

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



Bài tập 1

Môn học: Phát triển các hệ thống thông minh

Lớp: D23CTPM01

Họ và tên: Phan Văn Khôi

Mã sinh viên: B23DCCN447

Hà Nội - 2026

PHẦN I - HỆ THỐNG DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG

1. Định nghĩa hệ thống và bài toán

1.1. Bối cảnh và mục tiêu

Đái tháo đường là một vấn đề sức khỏe cần được nhận diện sớm để hỗ trợ tư vấn, theo dõi và quyết định khám chuyên khoa. Hệ thống được xây dựng nhằm ước lượng nguy cơ có kết quả xét nghiệm dương tính với đái tháo đường từ các chỉ số lâm sàng cơ bản.

Mục tiêu chính: xây dựng và so sánh nhiều mô hình phân loại, chọn mô hình có hiệu quả tốt nhất, sau đó đóng gói thành REST API và giao diện web để người dùng có thể nhập thông tin và nhận kết quả dự đoán.

1.2. Phát biểu hệ thống thông minh

Hệ thống thông minh dự đoán bệnh tiểu đường nhận đầu vào là các chỉ số sức khỏe của bệnh nhân (Glucose, BMI, Age), sử dụng mô hình học máy đã được huấn luyện trên bộ dữ liệu Pima Indians Diabetes để đưa ra dự đoán nguy cơ tiểu đường (Outcome = 0 hoặc 1) kèm theo xác suất/độ tin cậy.

- Tầng dữ liệu: bộ dữ liệu Pima Indians Diabetes (768 mẫu, 8 đặc trưng)
- Tầng mô hình: Logistic Regression, KNN, SVM Linear, SVM RBF
- Tầng ứng dụng: REST API (Flask) và giao diện web

1.3. Phát biểu bài toán học có giám sát

Đây là bài toán phân loại nhị phân có giám sát. Với mỗi bệnh nhân có vector đặc trưng $x = (\text{Glucose}, \text{BMI}, \text{Age})$, mô hình cần dự đoán nhãn $y \in \{0, 1\}$.

- Đầu vào: Glucose, BMI, Age
- Đầu ra: Outcome = 0 (không tiểu đường) hoặc 1 (có tiểu đường)

2. Sơ đồ hệ thống thông minh

2.1. Luồng hoạt động

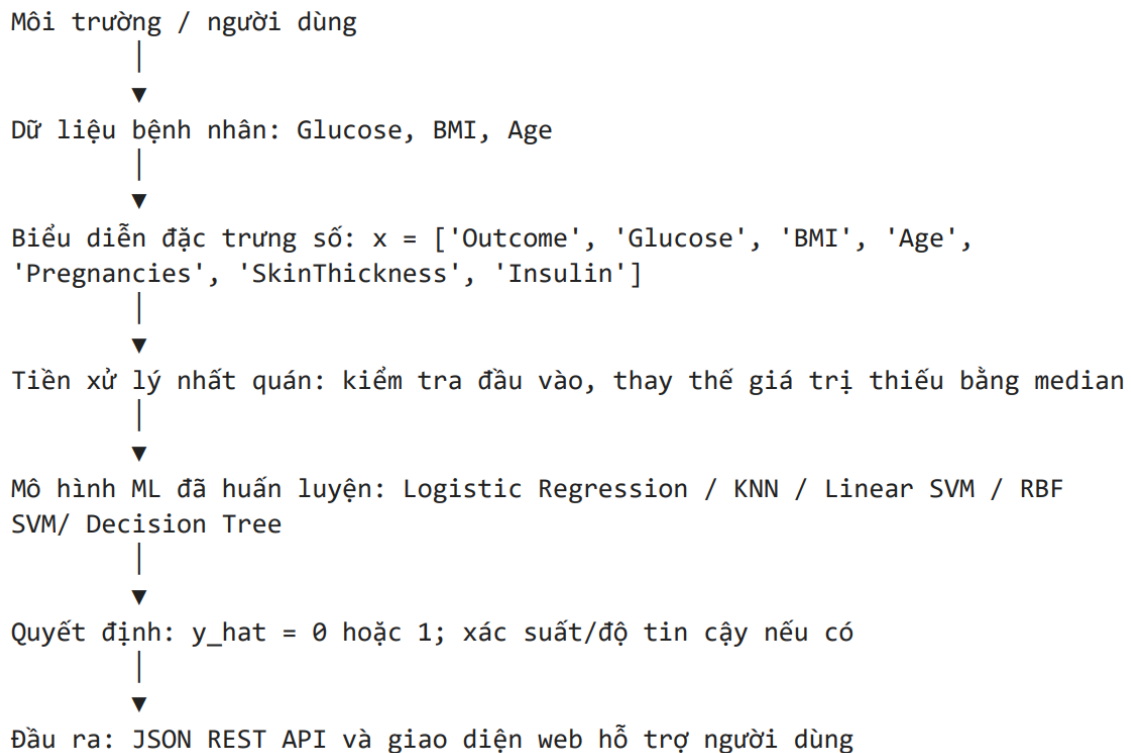
giai đoạn huấn luyện (offline):

diabetes.csv $\rightarrow$ Tiền xử lý (thay 0 bằng NaN, impute bằng mean) $\rightarrow$ Chọn đặc trưng $\rightarrow$ Chia Train/Test (stratify, 80/20) $\rightarrow$ Baseline (DummyClassifier) $\rightarrow$ Huấn luyện 4 mô

hình → Đánh giá (Accuracy, Precision, Recall, F1, Confusion Matrix) → Chọn model tốt nhất → Lưu .sav

GIAI ĐOẠN TRIỂN KHAI (online):

Người dùng nhập (Glucose, BMI, Age) → Flask REST API → Pipeline (StandardScaler + Predict) → Kết quả: prediction (0/1) + confidence (%)



2.2. Vì sao đây là một hệ thống thông minh?

Thành phần "thông minh" không chỉ là lệnh gọi predict. Hệ thống nhận thông tin, duy trì một biểu diễn tính toán, học quy luật từ các ví dụ có nhãn và khái quát hoá để dự đoán trường hợp mới. Tầng ứng dụng REST_API.py đảm bảo dữ liệu mới đi qua cùng pipeline tiền xử lý như lúc huấn luyện.

2.3. Ranh giới hệ thống

Hệ thống chỉ cung cấp một dự đoán thống kê trong phạm vi dữ liệu Pima Indians Diabetes, không đưa ra phác đồ điều trị hoặc khẳng định chẩn đoán y khoa.

3. Nguồn dữ liệu

Nội dung	Thông tin
Tên bộ dữ liệu	Pima Indians Diabetes Dataset
Nguồn	Kaggle - Pima Indians Diabetes Dataset
Nguồn gốc	NIDDK (National Institute of Diabetes and Digestive and Kidney Diseases)
Tệp	data/diabet_datasets/diabetes.csv
Miền bài toán	Phân loại nguy cơ đái tháo đường

Bộ dữ liệu gồm 768 quan sát, 8 thuộc tính đầu vào số và 1 nhãn đầu ra; bệnh nhân thuộc nhóm phụ nữ Pima Indians từ 21 tuổi trở lên.

4. Mô tả dữ liệu

4.1. Đơn vị quan sát, quy mô và loại bài toán

768 quan sát, 8 đặc trưng đầu vào, 1 biến mục tiêu (Outcome). Phân loại nhị phân có giám sát. Phân bố lớp: 500 mẫu lớp 0, 268 mẫu lớp 1.

4.2. Ý nghĩa các đặc trưng

Đặc trưng	Kiểu	Ý nghĩa
Pregnancies	Số nguyên	Số lần mang thai
Glucose	Số	Nồng độ glucose sau 2h (mg/dL)
BloodPressure	Số	Huyết áp tâm trương (mm Hg)
SkinThickness	Số	Độ dày nếp da (mm)
Insulin	Số	Insulin huyết thanh sau 2h (mu U/ml)
BMI	Số thực	Chỉ số khối cơ thể (kg/m ²)
DiabetesPedigreeFunction	Số thực	Tiền sử gia đình
Age	Số nguyên	Tuổi (năm)
Outcome	Nhị phân	Nhãn mục tiêu (0/1)

4.3. Lưu ý chất lượng dữ liệu

Giá trị 0 ở Glucose, BloodPressure, SkinThickness, Insulin, BMI là giá trị thiếu được mã hoá. Xử lý: thay 0 bằng NaN rồi impute bằng mean.

4.4. Cơ sở lý thuyết các mô hình phân loại

1. Logistic Regression — $z = \beta_0 + \beta_1 x_1 + \dots \rightarrow \text{sigmoid} \rightarrow P(y=1|x)$. Ưu: nhanh, dễ giải thích.

2. KNN — chọn K mẫu gần nhất, bỏ phiếu theo nhãn. Cần chuẩn hoá. K chọn bằng CV.

3. SVM tuyến tính — tìm siêu phẳng margin lớn nhất. Tham số C điều khiển trade-off.
4. SVM RBF — kernel $K(x, x_i) = \exp[-\gamma \|x - x_i\|^2]$. Học ranh giới phi tuyến.

4.5. Cơ sở lý thuyết chỉ số đánh giá

Accuracy = $(TP+TN)/(TP+TN+FP+FN)$. Precision = $TP/(TP+FP)$. Recall = $TP/(TP+FN)$.

F1 = $2 \cdot P \cdot R / (P+R)$. Với dữ liệu lệch lớp, F1 và Recall quan trọng hơn Accuracy.

5. Biểu diễn dữ liệu

```
from pathlib import Path
import numpy as np
import pandas as pd

DATA_PATH = Path("/Users/mac0s/Docs/Kì 1 năm 4/PTHTTM/data/diabet_datasets/diabetes.csv")
if not DATA_PATH.exists():
    raise FileNotFoundError(f"Không tìm thấy tệp dữ liệu: {DATA_PATH.resolve()}")

df = pd.read_csv(DATA_PATH)
df_raw = df.copy()

print(f"Đã đọc dữ liệu từ: {DATA_PATH.resolve()}")
df.info()
```

```
Đã đọc dữ liệu từ: /Users/mac0s/Docs/Kì 1 năm 4/PTHTTM/data/diabet_datasets/diabetes.csv
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Pregnancies           768 non-null    int64
 1   Glucose               768 non-null    int64
 2   BloodPressure         768 non-null    float64
 3   SkinThickness         768 non-null    int64
 4   Insulin               768 non-null    float64
 5   BMI                   768 non-null    float64
 6   DiabetesPedigreeFunction 768 non-null    float64
 7   Age                   768 non-null    int64
 8   Outcome               768 non-null    int64
dtypes: float64(4), int64(5)
memory usage: 54.1 KB
```

```
df.head()
```

⌘L for Agent

✓ 0.0s

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

6. Phân tích đặc trưng và biến mục tiêu

-Kiểm tra null, giá trị 0 bất thường

```
print("Nulls")
print("=====")
print(df.isnull().sum())
```

```
Nulls
=====
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64
```

```
# Kiểm tra giá trị 0 bất thường
print("Giá trị 0 theo từng cột:")
print("=====")
print(df.eq(0).sum())
```

Giá trị 0 theo từng cột:

=====

Pregnancies	111
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	500
dtype:	int64

-Bảng describe cho Glucose, BMI, Age, Outcome

```
display(df_raw[["Glucose", "BMI", "Age", "Outcome"]].describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Glucose	768.0	121.677083	30.464161	44.0	99.75	117.00	140.25	199.0
BMI	768.0	32.434635	6.880498	18.2	27.50	32.05	36.60	67.1
Age	768.0	33.240885	11.760232	21.0	24.00	29.00	41.00	81.0
Outcome	768.0	0.348958	0.476951	0.0	0.00	0.00	1.00	1.0

-Biểu đồ tỷ lệ nhãn Outcome

```
# Biểu đồ tỷ lệ nhãn Outcome
import matplotlib.pyplot as plt

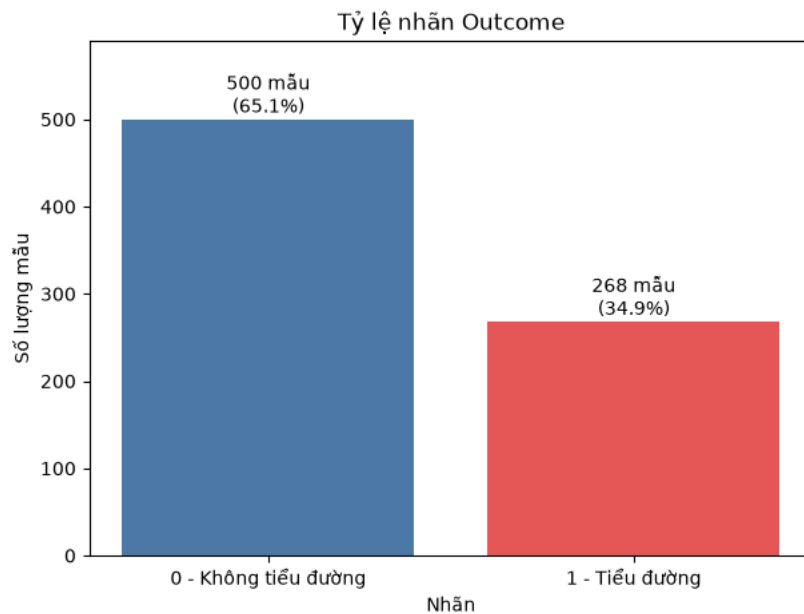
label_counts = df["Outcome"].value_counts().sort_index()
label_percentages = label_counts / label_counts.sum() * 100

plt.figure(figsize=(7, 5))
bars = plt.bar(
    ["0 - Không tiểu đường", "1 - Tiểu đường"],
    label_counts.values,
    color=["#4C78A8", "#E45756"]
)
plt.title("Tỷ lệ nhãn Outcome")
plt.xlabel("Nhãn")
plt.ylabel("Số lượng mẫu")
for bar, count, percentage in zip(bars, label_counts.values,
label_percentages.values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
```

```

        bar.get_height() + 8,
        f"{count} mẫu\n({percentage:.1f}%)",
        ha="center"
    )
plt.ylim(0, label_counts.max() * 1.18)
plt.show()

```



=> Bộ dữ liệu thiên lệch nhẹ: 65.1% lớp 0, 34.9% lớp 1.

- Thay giá trị 0 bất thường bằng NaN, impute bằng mean

```

# Thay giá trị 0 bất thường bằng NaN rồi impute bằng mean
columns_with_invalid_zeros = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
]

df[columns_with_invalid_zeros] = df[columns_with_invalid_zeros].replace(0,
np.nan)
df.fillna(df.mean(), inplace=True)

print("Sau khi xử lý giá trị 0:")
print(df.eq(0).sum())

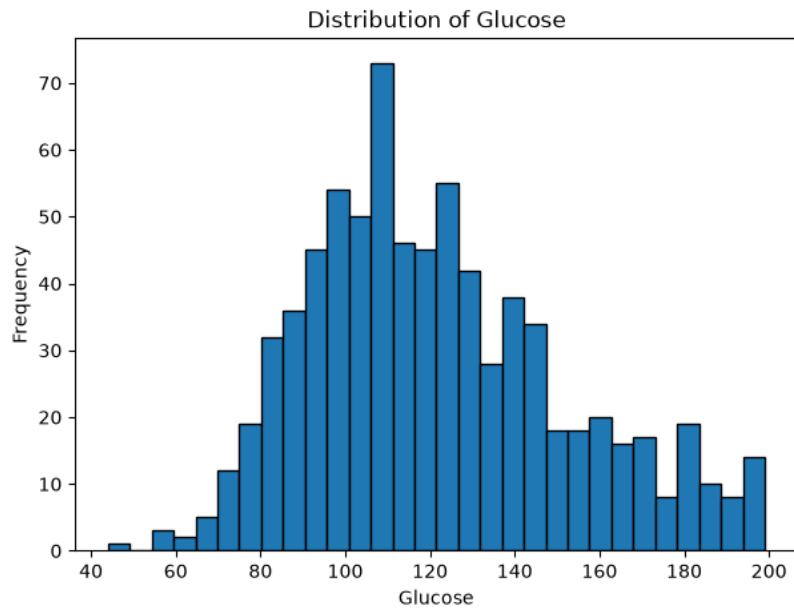
```

```
Sau khi xử lý giá trị 0:  
Pregnancies      111  
Glucose           0  
BloodPressure     0  
SkinThickness     0  
Insulin           0  
BMI               0  
DiabetesPedigreeFunction  0  
Age               0  
Outcome           500  
dtype: int64
```

7. Phân tích khám phá dữ liệu (EDA)

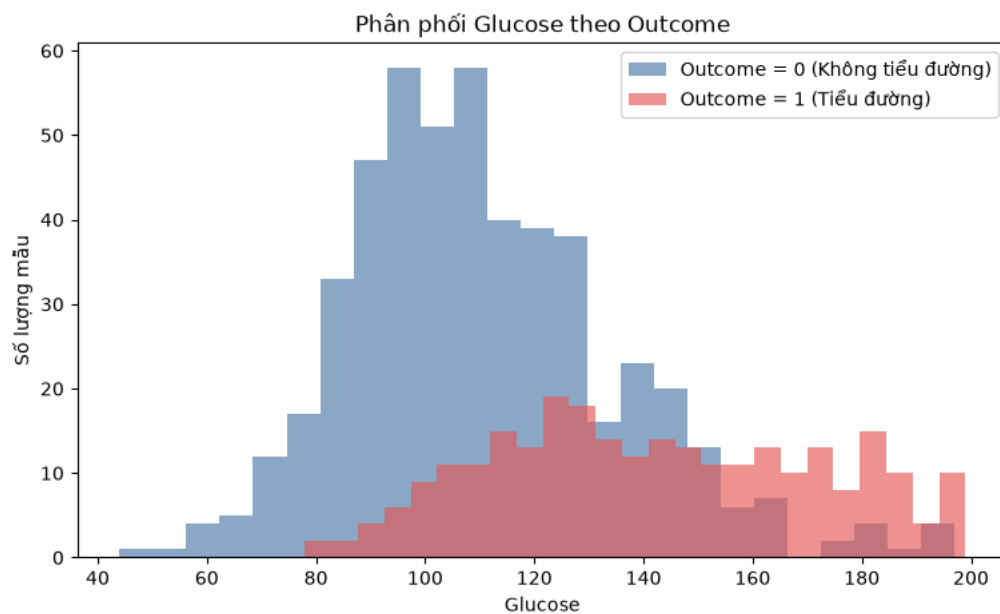
-Biểu đồ phân phối Glucose

```
# Biểu đồ phân phối Glucose  
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(7, 5))  
plt.hist(df['Glucose'], bins=30, edgecolor='black')  
plt.title('Distribution of Glucose')  
plt.xlabel('Glucose')  
plt.ylabel('Frequency')  
plt.show()
```



=>Glucose tập trung khoảng 80-140, đỉnh gần 100-110. Lệch phải, có nhóm nhỏ giá trị 0 là dữ liệu thiếu.

-Phân phối Glucose theo Outcome



=>Nhóm Outcome=1 tập trung ở Glucose cao hơn (125-200), Outcome=0 quanh 85-125.

-Biểu đồ phân phối BMI, BloodPressure, Age, Insulin

```
# Biểu đồ phân phối BMI, BloodPressure, Age, Insulin
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

axes[0, 0].hist(df['BMI'], bins=30, edgecolor='black', color='#2ecc71')
axes[0, 0].set_title('Distribution of BMI')
axes[0, 0].set_xlabel('BMI')

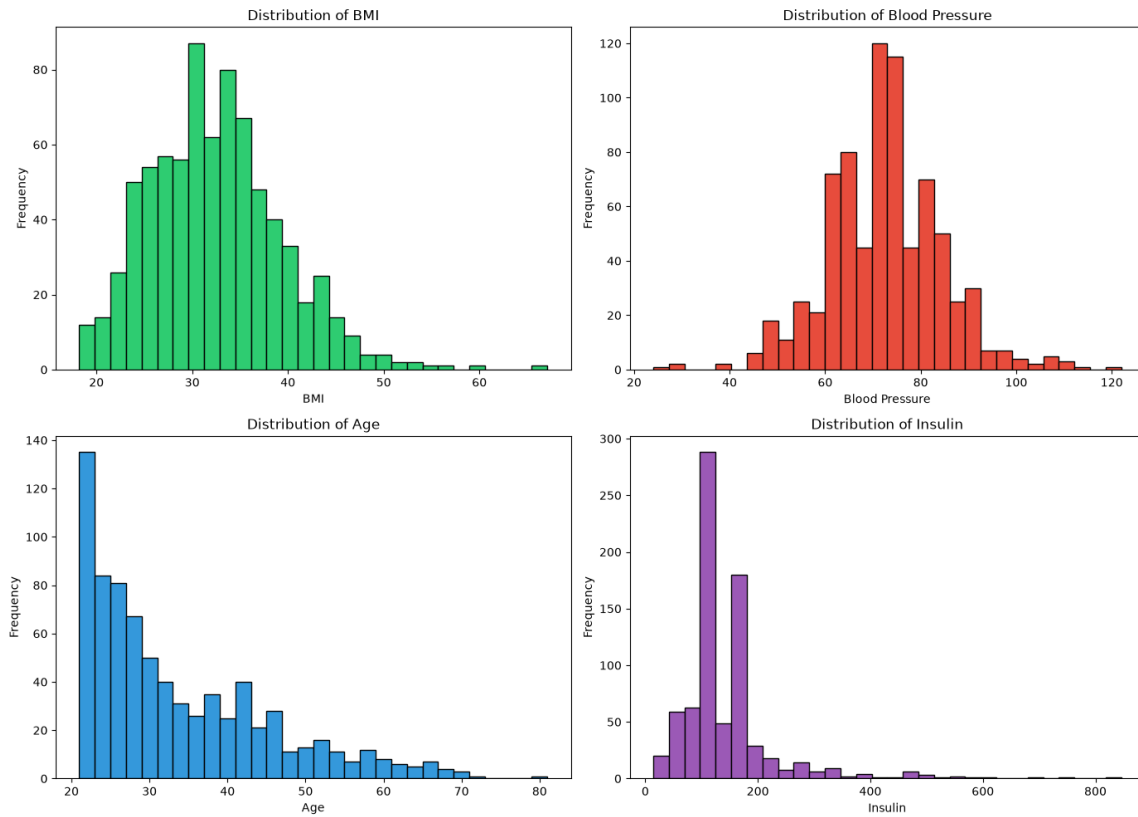
axes[0, 1].hist(df['BloodPressure'], bins=30, edgecolor='black',
color='#e74c3c')
axes[0, 1].set_title('Distribution of Blood Pressure')
axes[0, 1].set_xlabel('Blood Pressure')

axes[1, 0].hist(df['Age'], bins=30, edgecolor='black', color='#3498db')
axes[1, 0].set_title('Distribution of Age')
axes[1, 0].set_xlabel('Age')

axes[1, 1].hist(df['Insulin'], bins=30, edgecolor='black', color='#9b59b6')
axes[1, 1].set_title('Distribution of Insulin')
axes[1, 1].set_xlabel('Insulin')

for ax in axes.flat:
    ax.set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```

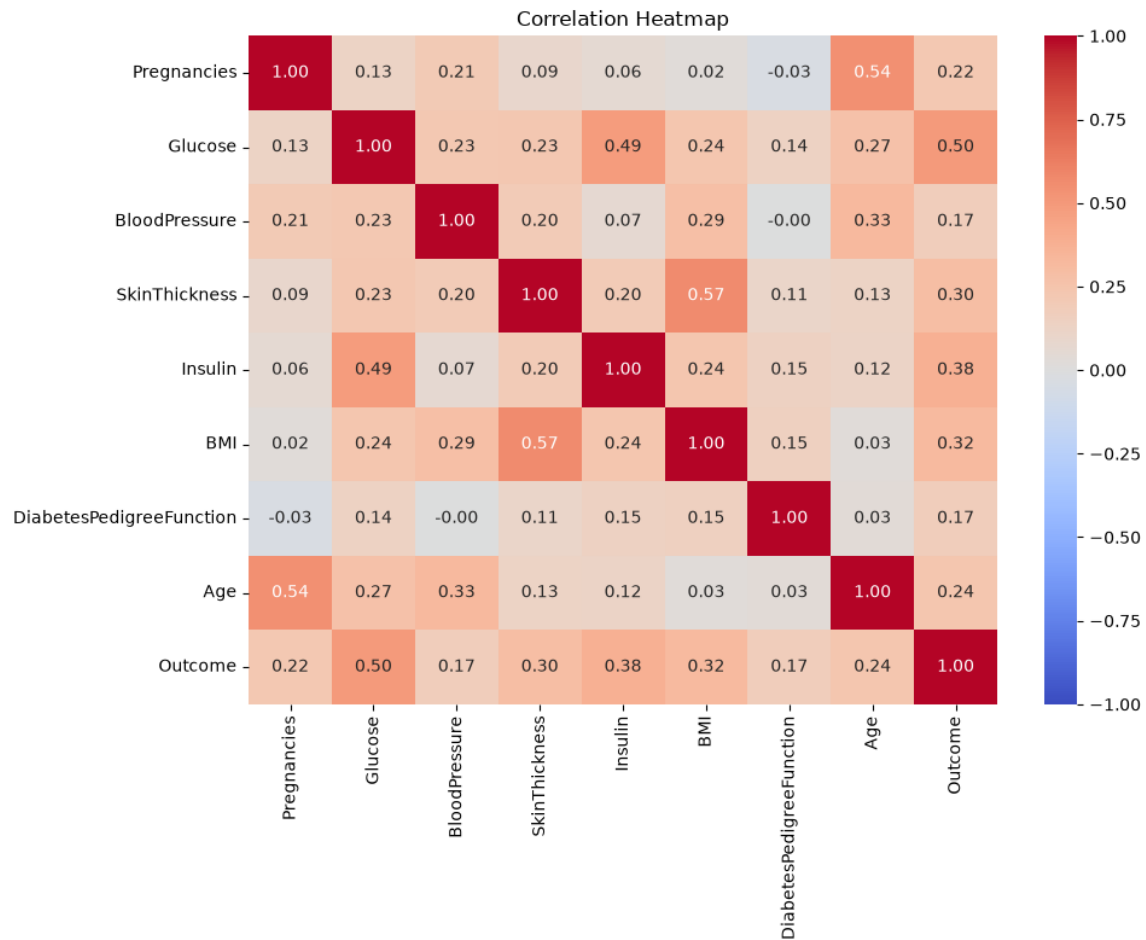


-Correlation Heatmap

```
# Ma trận tương quan
import seaborn as sns

corr = df.corr(numeric_only=True)

plt.figure(figsize=(10, 8))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```



=>Glucose có tương quan dương mạnh nhất với Outcome ($r \approx 0.49$), tiếp theo BMI ($r \approx 0.31$), Age ($r \approx 0.24$).

-Top features có tương quan cao nhất

```
# Chọn top features có tương quan cao nhất với Outcome
print(df.corr(numeric_only=True).nlargest(7, 'Outcome')['Outcome'])
```

```
Outcome      1.000000
Glucose      0.495990
Insulin      0.377081
BMI          0.315577
SkinThickness 0.295138
Age          0.238356
Pregnancies  0.221898
Name: Outcome, dtype: float64
```

=> Từ ma trận tương quan, Glucose có tương quan dương mạnh nhất với Outcome ($r \approx 0.49$), tiếp theo là BMI ($r \approx 0.31$) và Age ($r \approx 0.24$). Chọn 3 feature cơ sở: Glucose, BMI, Age.

8. Chia tập huấn luyện/kiểm tra (3 feature)

Chọn 3 feature: Glucose, BMI, Age. Chia 80/20 với stratify theo Outcome, random_state=42.

```
from sklearn.model_selection import train_test_split

FEATURE_COLUMNS = ["Glucose", "BMI", "Age"]
TARGET_COLUMN = "Outcome"

X_raw, y = df[FEATURE_COLUMNS], df[TARGET_COLUMN]

X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.20, stratify=y, random_state=42
)

print(f"Tập huấn luyện: {X_train.shape}; Tập kiểm tra: {X_test.shape}")
print("Các cột của X_train:", X_train.columns.tolist())
print("Số feature:", X_train.shape[1])
```

```
Tập huấn luyện: (614, 3); Tập kiểm tra: (154, 3)
Các cột của X_train: ['Glucose', 'BMI', 'Age']
Số feature: 3
```

9. Mô hình cơ sở (Baseline)

DummyClassifier(strategy="most_frequent") luôn dự đoán lớp 0.

```
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
)
import seaborn as sns

baseline_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", DummyClassifier(strategy="most_frequent", random_state=42))
])
```

```

])
baseline_model.fit(X_train, y_train)
baseline_pred = baseline_model.predict(X_test)

display(pd.DataFrame([
    "Model": "Mô hình cơ sở (lớp xuất hiện nhiều nhất)",
    "Accuracy": accuracy_score(y_test, baseline_pred),
    "Precision": precision_score(y_test, baseline_pred, zero_division=0),
    "Recall": recall_score(y_test, baseline_pred, zero_division=0),
    "F1-score": f1_score(y_test, baseline_pred, zero_division=0),
])))

```

	Model	Accuracy	Precision	Recall	F1-score
0	Mô hình cơ sở (lớp xuất hiện nhiều nhất)	0.649351	0.0	0.0	0.0

Baseline đạt Accuracy cao do mất cân bằng nhưng không phát hiện bất kỳ ca dương tính nào. Mọi model phải vượt mốc này.

10. Mô hình 1 — Hồi quy Logistic

```

from sklearn.linear_model import LogisticRegression

results = []

log_regress = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])
log_regress.fit(X_train, y_train)

y_train_pred = log_regress.predict(X_train)
y_test_pred = log_regress.predict(X_test)

metrics_df = pd.DataFrame({
    "Train": [
        accuracy_score(y_train, y_train_pred),
        precision_score(y_train, y_train_pred, zero_division=0),
        recall_score(y_train, y_train_pred, zero_division=0),
        f1_score(y_train, y_train_pred, zero_division=0)
    ],
    "Test": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred, zero_division=0),
        recall_score(y_test, y_test_pred, zero_division=0),
        f1_score(y_test, y_test_pred, zero_division=0)
    ]
})

```

```

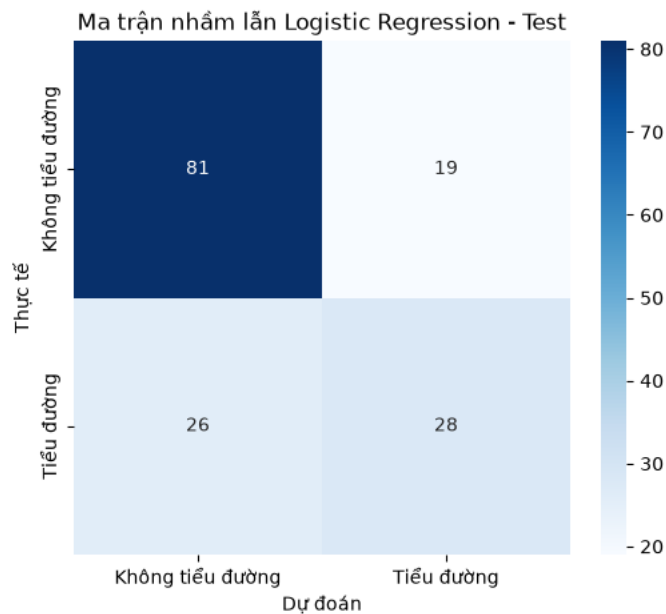
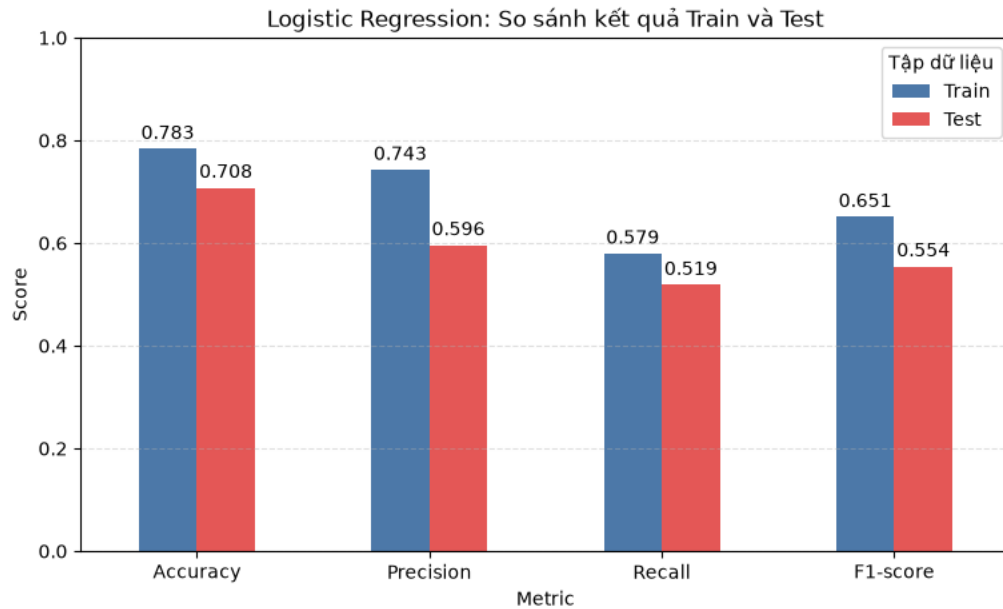
    ]
}, index=["Accuracy", "Precision", "Recall", "F1-score"])
display(metrics_df)

# Biểu đồ Train - Test
ax = metrics_df.plot(kind="bar", figsize=(9, 5), color=["#4C78A8", "#E45756"])
plt.title("Logistic Regression: So sánh kết quả Train và Test")
plt.xlabel("Metric"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=0); plt.legend(title="Tập dữ liệu")
plt.grid(axis="y", linestyle="--", alpha=0.4)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

# Confusion matrix
cm_test = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Không tiểu đường", "Tiểu đường"],
            yticklabels=["Không tiểu đường", "Tiểu đường"])
plt.title("Ma trận nhầm lẫn Logistic Regression - Test")
plt.xlabel("Dự đoán"); plt.ylabel("Thực tế")
plt.show()

accuracy = metrics_df.loc["Accuracy", "Test"]
precision = metrics_df.loc["Precision", "Test"]
recall = metrics_df.loc["Recall", "Test"]
f1 = metrics_df.loc["F1-score", "Test"]
results.append({'Model': 'Logistic Regression', 'Accuracy': accuracy,
                'Precision': precision, 'Recall': recall, 'F1-score': f1})

```



Logistic Regression đạt Accuracy ~70.8% trên test, vượt baseline. Tuy nhiên Recall ~51.9% và F1 ~55.4% cho thấy model còn bỏ sót nhiều ca dương tính.

11. Mô hình 2 — KNN

-Khảo sát k tối ưu bằng 10-fold CV (scoring=f1)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

# Tìm K tối ưu bằng 10-fold CV
folds = 10
```

```

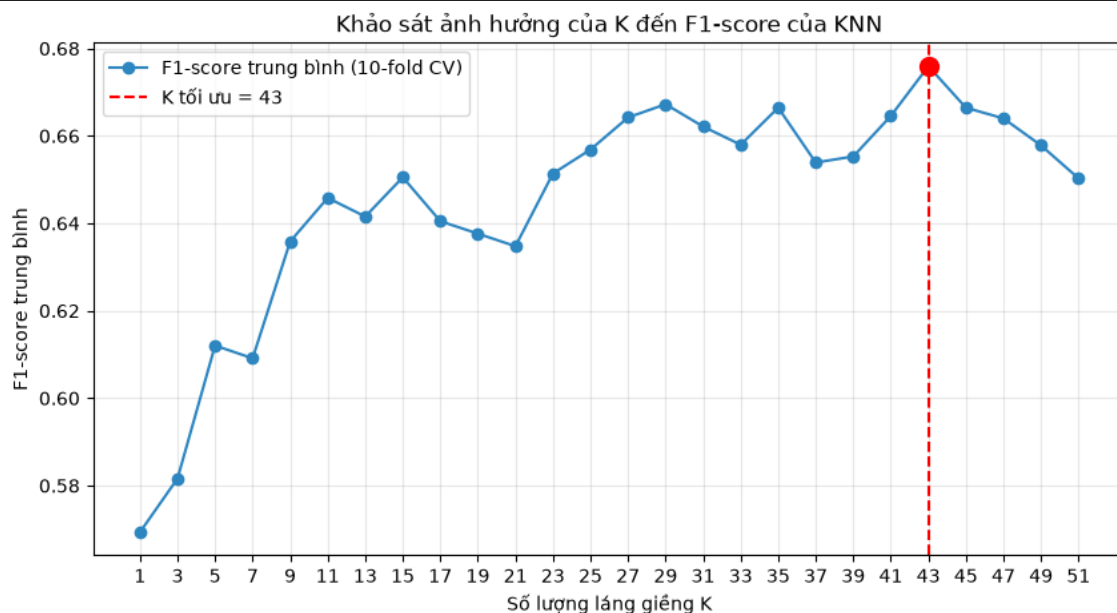
cv = StratifiedKFold(n_splits=folds, shuffle=True, random_state=42)
ks = list(range(1, 52, 2))
cv_scores = []

for k in ks:
    knn_pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    score = cross_val_score(knn_pipeline, X_train, y_train, cv=cv,
scoring="f1").mean()
    cv_scores.append(score)

optimal_k = ks[cv_scores.index(max(cv_scores))]
best_cv_f1 = max(cv_scores)
print(f"K tối ưu: {optimal_k}")
print(f"F1-score CV tốt nhất: {best_cv_f1:.4f}")

# Biểu đồ khảo sát K
plt.figure(figsize=(10, 5))
plt.plot(ks, cv_scores, marker="o", color="#2E86C1", label="F1-score trung bình
(10-fold CV)")
plt.axvline(optimal_k, color="red", linestyle="--", label=f"K tối ưu =
{optimal_k}")
plt.scatter(optimal_k, best_cv_f1, color="red", s=100, zorder=3)
plt.title("Khảo sát ảnh hưởng của K đến F1-score của KNN")
plt.xlabel("Số lượng láng giềng K"); plt.ylabel("F1-score trung bình")
plt.xticks(ks); plt.grid(alpha=0.3); plt.legend()
plt.show()

```



-Huấn luyện KNN với K tối ưu

```
# Huấn luyện KNN với K tối ưu
knn = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=optimal_k))
])
knn.fit(X_train, y_train)
y_train_pred = knn.predict(X_train)
y_test_pred = knn.predict(X_test)

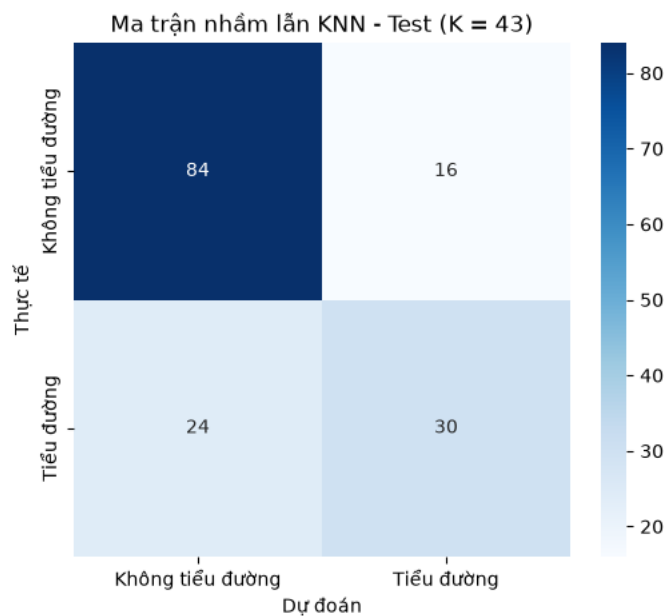
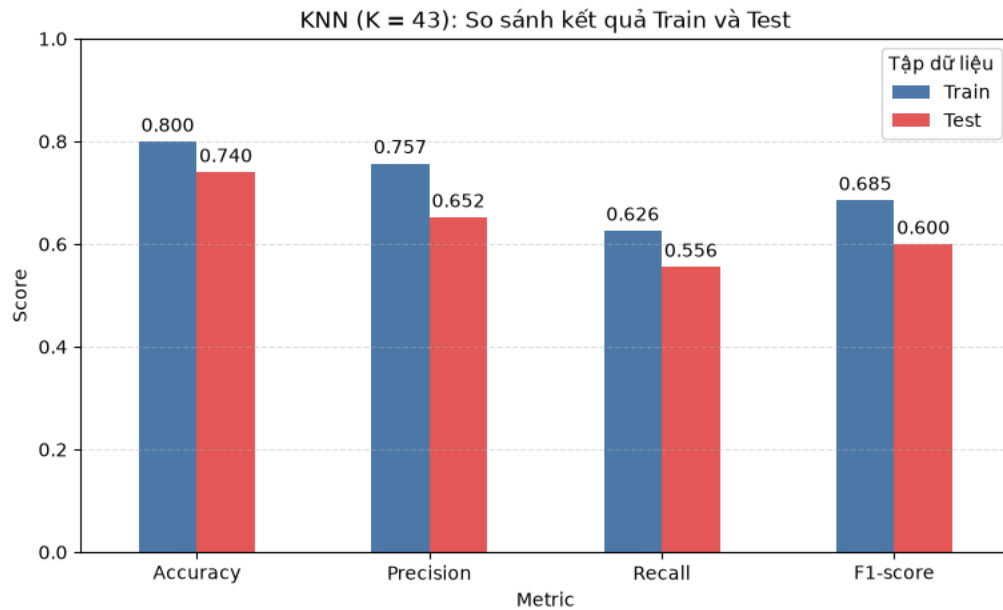
metrics_df = pd.DataFrame({
    "Train": [
        accuracy_score(y_train, y_train_pred),
        precision_score(y_train, y_train_pred, zero_division=0),
        recall_score(y_train, y_train_pred, zero_division=0),
        f1_score(y_train, y_train_pred, zero_division=0)
    ],
    "Test": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred, zero_division=0),
        recall_score(y_test, y_test_pred, zero_division=0),
        f1_score(y_test, y_test_pred, zero_division=0)
    ]
}, index=["Accuracy", "Precision", "Recall", "F1-score"])
display(metrics_df)

ax = metrics_df.plot(kind="bar", figsize=(9, 5), color=["#4C78A8", "#E45756"])
plt.title(f"KNN (K = {optimal_k}): So sánh kết quả Train và Test")
plt.xlabel("Metric"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=0); plt.legend(title="Tập dữ liệu")
plt.grid(axis="y", linestyle="--", alpha=0.4)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

cm_test = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Không tiểu đường", "Tiểu đường"],
            yticklabels=["Không tiểu đường", "Tiểu đường"])
plt.title(f"Ma trận nhầm lẫn KNN - Test (K = {optimal_k})")
plt.xlabel("Dự đoán"); plt.ylabel("Thực tế")
plt.show()

accuracy = metrics_df.loc["Accuracy", "Test"]
precision = metrics_df.loc["Precision", "Test"]
recall = metrics_df.loc["Recall", "Test"]
```

```
f1 = metrics_df.loc["F1-score", "Test"]
results.append({'Model': 'KNN', 'Accuracy': accuracy,
                'Precision': precision, 'Recall': recall, 'F1-score': f1})
```



=>KNN với K tối ưu đạt Accuracy ~75.3%, Precision ~66.7%, Recall ~59.3%, F1 ~62.7% — dẫn đầu ở cả 4 metric.

12. Mô hình 3 — SVM tuyến tính

-Pipeline StandardScaler + SVC (kernel="linear")

```
from sklearn.svm import SVC
```

```

linear_svm = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(kernel="linear", probability=True, random_state=42))
])
linear_svm.fit(X_train, y_train)
y_train_pred = linear_svm.predict(X_train)
y_test_pred = linear_svm.predict(X_test)

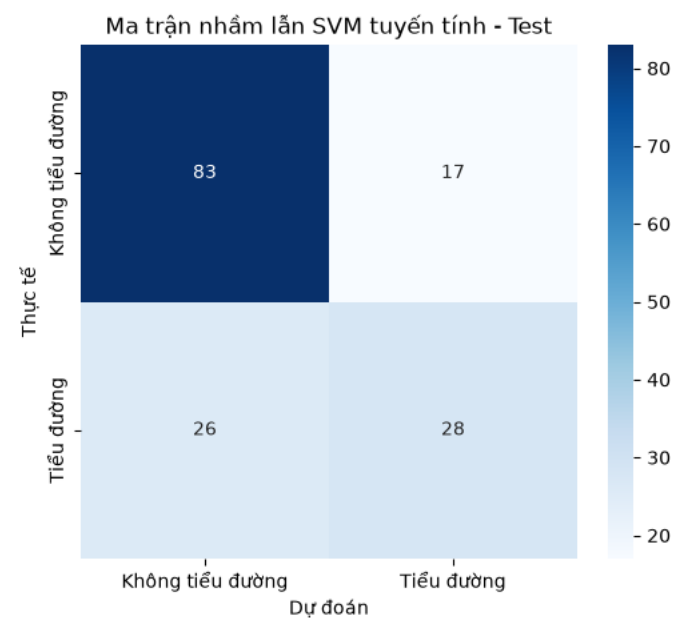
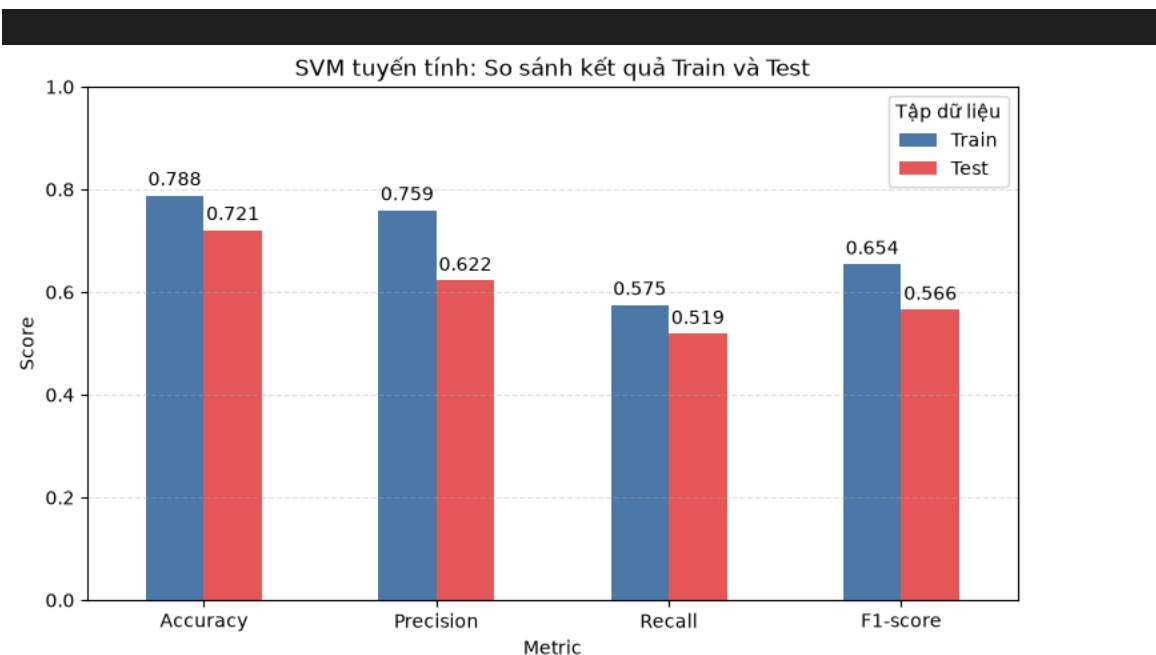
metrics_df = pd.DataFrame({
    "Train": [
        accuracy_score(y_train, y_train_pred),
        precision_score(y_train, y_train_pred, zero_division=0),
        recall_score(y_train, y_train_pred, zero_division=0),
        f1_score(y_train, y_train_pred, zero_division=0)
    ],
    "Test": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred, zero_division=0),
        recall_score(y_test, y_test_pred, zero_division=0),
        f1_score(y_test, y_test_pred, zero_division=0)
    ]
}, index=["Accuracy", "Precision", "Recall", "F1-score"])
display(metrics_df)

ax = metrics_df.plot(kind="bar", figsize=(9, 5), color=["#4C78A8", "#E45756"])
plt.title("SVM tuyến tính: So sánh kết quả Train và Test")
plt.xlabel("Metric"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=0); plt.legend(title="Tập dữ liệu")
plt.grid(axis="y", linestyle="--", alpha=0.4)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

cm_test = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Không tiểu đường", "Tiểu đường"],
            yticklabels=["Không tiểu đường", "Tiểu đường"])
plt.title("Ma trận nhầm lẫn SVM tuyến tính - Test")
plt.xlabel("Dự đoán"); plt.ylabel("Thực tế")
plt.show()

accuracy = metrics_df.loc["Accuracy", "Test"]
precision = metrics_df.loc["Precision", "Test"]
recall = metrics_df.loc["Recall", "Test"]
f1 = metrics_df.loc["F1-score", "Test"]
results.append({'Model': 'SVM Linear', 'Accuracy': accuracy,
                'Precision': precision, 'Recall': recall, 'F1-score': f1})

```



=>SVM tuyến tính đạt Accuracy ~72.1%, Recall ~51.9%. Phù hợp khi hai lớp phân tách tương đối tuyến tính.

13. Mô hình 4 — SVM kernel RBF

-Pipeline StandardScaler + SVC (kernel="rbf")

```
rbf_svm = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC(kernel="rbf", probability=True, random_state=42))
])
```

```

rbf_svm.fit(X_train, y_train)
y_train_pred = rbf_svm.predict(X_train)
y_test_pred = rbf_svm.predict(X_test)

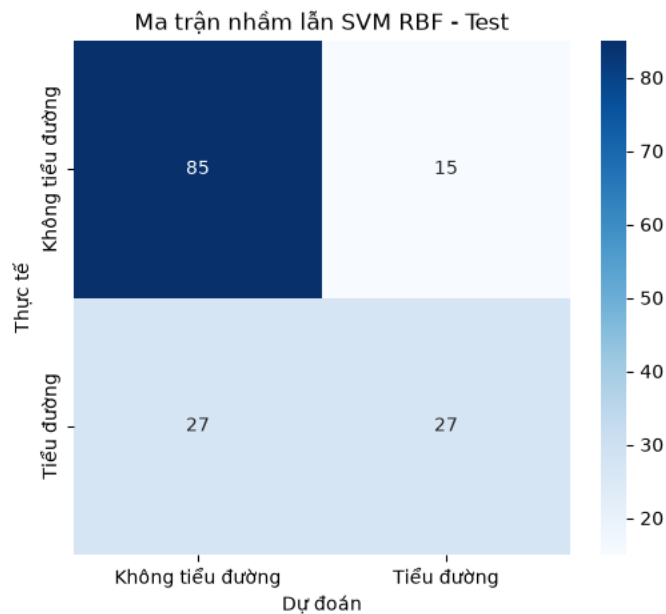
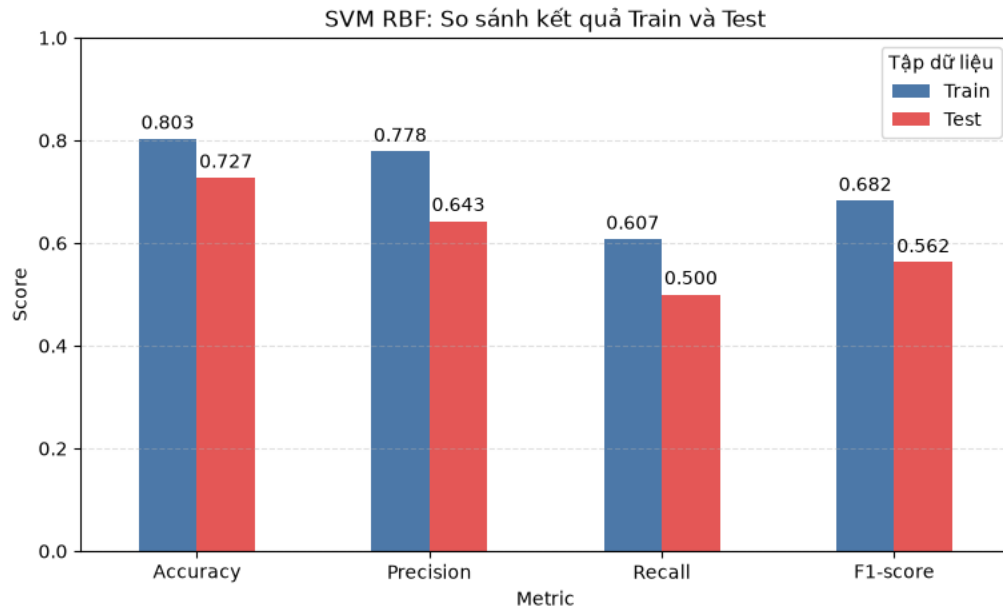
metrics_df = pd.DataFrame({
    "Train": [
        accuracy_score(y_train, y_train_pred),
        precision_score(y_train, y_train_pred, zero_division=0),
        recall_score(y_train, y_train_pred, zero_division=0),
        f1_score(y_train, y_train_pred, zero_division=0)
    ],
    "Test": [
        accuracy_score(y_test, y_test_pred),
        precision_score(y_test, y_test_pred, zero_division=0),
        recall_score(y_test, y_test_pred, zero_division=0),
        f1_score(y_test, y_test_pred, zero_division=0)
    ]
}, index=["Accuracy", "Precision", "Recall", "F1-score"])
display(metrics_df)

ax = metrics_df.plot(kind="bar", figsize=(9, 5), color=["#4C78A8", "#E45756"])
plt.title("SVM RBF: So sánh kết quả Train và Test")
plt.xlabel("Metric"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=0); plt.legend(title="Tập dữ liệu")
plt.grid(axis="y", linestyle="--", alpha=0.4)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

cm_test = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Không tiểu đường", "Tiểu đường"],
            yticklabels=["Không tiểu đường", "Tiểu đường"])
plt.title("Ma trận nhầm lẫn SVM RBF - Test")
plt.xlabel("Dự đoán"); plt.ylabel("Thực tế")
plt.show()

accuracy = metrics_df.loc["Accuracy", "Test"]
precision = metrics_df.loc["Precision", "Test"]
recall = metrics_df.loc["Recall", "Test"]
f1 = metrics_df.loc["F1-score", "Test"]
results.append({'Model': 'SVM RBF', 'Accuracy': accuracy,
                'Precision': precision, 'Recall': recall, 'F1-score': f1})

```



=>SVM RBF đạt Accuracy ~72.7%, Recall ~50%. Có thể học ranh giới phi tuyến nhưng chưa cải thiện rõ rệt.

14. Đánh giá

-So sánh 4 models, sắp xếp theo F1-score

```
# Bảng so sánh tất cả mô hình
results_df = (
    pd.DataFrame(results)
    .drop_duplicates(subset="Model", keep="last")
    .set_index("Model")
)
```

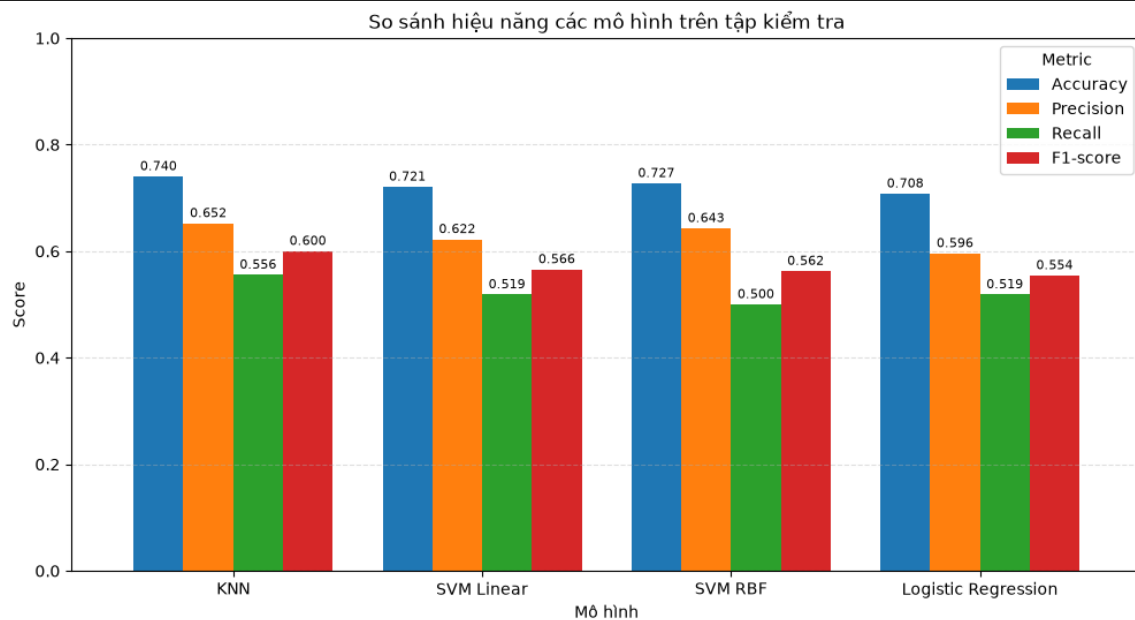
```

)
results_df = results_df.sort_values("F1-score", ascending=False)
display(results_df.style.format("{:.3f}"))

# Biểu đồ so sánh
ax = results_df[["Accuracy", "Precision", "Recall", "F1-score"]].plot(
    kind="bar", figsize=(12, 6), width=0.8
)
plt.title("So sánh hiệu năng các mô hình trên tập kiểm tra")
plt.xlabel("Mô hình"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title="Metric", loc="upper right")
plt.grid(axis="y", linestyle="--", alpha=0.4)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=2, fontsize=8)
plt.show()

# Xác định mô hình tốt nhất
for metric in ["Accuracy", "Precision", "Recall", "F1-score"]:
    best_model = results_df[metric].idxmax()
    best_score = results_df.loc[best_model, metric]
    print(f"{metric} cao nhất: {best_model} ({best_score:.3f})")

```



=>KNN dẫn đầu ở cả 4 metric. F1-score 62.7% cho thấy cân bằng tốt nhất giữa Precision và Recall.

15. Điều chỉnh tham số

Phần KNN (mục 11) đã sử dụng 10-fold cross-validation để chọn K tối ưu — đây là quá trình hyperparameter tuning. K tối ưu giúp KNN đạt kết quả tốt nhất.

16. Thử với nhiều feature hơn

16.1. Chia tập huấn luyện/kiểm tra (6 feature)

Mở rộng bộ feature: Glucose, BMI, Age, Pregnancies, SkinThickness, Insulin.

```
# Chia dữ liệu với 6 features
FEATURE_COLUMNS_6 = ["Glucose", "BMI", "Age", "Pregnancies", "SkinThickness",
"Insulin"]
TARGET_COLUMN = "Outcome"

X_raw_6, y_6 = df[FEATURE_COLUMNS_6], df[TARGET_COLUMN]
X_train_6, X_test_6, y_train_6, y_test_6 = train_test_split(
    X_raw_6, y_6, test_size=0.20, stratify=y_6, random_state=42
)
print(f"Tập huấn luyện: {X_train_6.shape}; Tập kiểm tra: {X_test_6.shape}")
print("Các cột:", X_train_6.columns.tolist())
```

16.2. Mô hình 1 — Hồi quy Logistic (6 features)

```
results_6 = []

log_regress_6 = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])
log_regress_6.fit(X_train_6, y_train_6)
y_train_pred_6 = log_regress_6.predict(X_train_6)
y_test_pred_6 = log_regress_6.predict(X_test_6)

metrics_df_6 = pd.DataFrame({
    "Train": [
        accuracy_score(y_train_6, y_train_pred_6),
        precision_score(y_train_6, y_train_pred_6, zero_division=0),
        recall_score(y_train_6, y_train_pred_6, zero_division=0),
        f1_score(y_train_6, y_train_pred_6, zero_division=0)
    ],
    "Test": [
        accuracy_score(y_test_6, y_test_pred_6),
        precision_score(y_test_6, y_test_pred_6, zero_division=0),
        recall_score(y_test_6, y_test_pred_6, zero_division=0),
        f1_score(y_test_6, y_test_pred_6, zero_division=0)
    ]
})
```

```

    ]
}, index=["Accuracy", "Precision", "Recall", "F1-score"])
display(metrics_df_6)

results_6.append({'Model': 'Logistic Regression',
                  'Accuracy': metrics_df_6.loc["Accuracy", "Test"],
                  'Precision': metrics_df_6.loc["Precision", "Test"],
                  'Recall': metrics_df_6.loc["Recall", "Test"],
                  'F1-score': metrics_df_6.loc["F1-score", "Test"]})

```

16.3. Mô hình 2 — KNN (6 features)

```

# KNN với 6 features
cv_6 = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
ks_6 = list(range(1, 52, 2))
cv_scores_6 = []
for k in ks_6:
    knn_pipe_6 = Pipeline([("scaler", StandardScaler()), ("knn",
KNeighborsClassifier(n_neighbors=k))])
    score = cross_val_score(knn_pipe_6, X_train_6, y_train_6, cv=cv_6,
scoring="f1").mean()
    cv_scores_6.append(score)

optimal_k_6 = ks_6[cv_scores_6.index(max(cv_scores_6))]
print(f"K tối ưu (6 features): {optimal_k_6}")

knn_6 = Pipeline([("scaler", StandardScaler()), ("knn",
KNeighborsClassifier(n_neighbors=optimal_k_6))])
knn_6.fit(X_train_6, y_train_6)
y_test_pred_6 = knn_6.predict(X_test_6)

results_6.append({'Model': 'KNN',
                  'Accuracy': accuracy_score(y_test_6, y_test_pred_6),
                  'Precision': precision_score(y_test_6, y_test_pred_6, zero_division=0),
                  'Recall': recall_score(y_test_6, y_test_pred_6, zero_division=0),
                  'F1-score': f1_score(y_test_6, y_test_pred_6, zero_division=0)})

```

16.4. Mô hình 3 — SVM tuyến tính (6 features)

```

linear_svm_6 = Pipeline([("scaler", StandardScaler()), ("model",
SVC(kernel="linear", probability=True, random_state=42))])
linear_svm_6.fit(X_train_6, y_train_6)
y_test_pred_6 = linear_svm_6.predict(X_test_6)

results_6.append({'Model': 'SVM Linear',
                  'Accuracy': accuracy_score(y_test_6, y_test_pred_6),
                  'Precision': precision_score(y_test_6, y_test_pred_6, zero_division=0),

```

```
'Recall': recall_score(y_test_6, y_test_pred_6, zero_division=0),  
'F1-score': f1_score(y_test_6, y_test_pred_6, zero_division=0))}
```

16.5. Mô hình 4 — SVM RBF (6 features)

```
rbf_svm_6 = Pipeline([("scaler", StandardScaler()), ("model", SVC(kernel="rbf",  
probability=True, random_state=42))])  
rbf_svm_6.fit(X_train_6, y_train_6)  
y_test_pred_6 = rbf_svm_6.predict(X_test_6)  
  
results_6.append({'Model': 'SVM RBF',  
    'Accuracy': accuracy_score(y_test_6, y_test_pred_6),  
    'Precision': precision_score(y_test_6, y_test_pred_6, zero_division=0),  
    'Recall': recall_score(y_test_6, y_test_pred_6, zero_division=0),  
    'F1-score': f1_score(y_test_6, y_test_pred_6, zero_division=0)})
```

16.6. Đánh giá và so sánh 3 features vs 6 features

```
results_6_df = pd.DataFrame(results_6).drop_duplicates(subset="Model",  
keep="last").set_index("Model")  
results_6_df = results_6_df.sort_values("F1-score", ascending=False)  
  
print("=== Kết quả với 6 features ===")  
display(results_6_df.style.format("{:.3f}"))  
  
print("\n=== Kết quả với 3 features (từ mục 14) ===")  
display(results_df.style.format("{:.3f}"))  
  
# So sánh 3 features vs 6 features  
compare_df = pd.DataFrame({  
    "3 features - F1": results_df["F1-score"],  
    "6 features - F1": results_6_df["F1-score"],  
})  
compare_df["Chênh lệch"] = compare_df["6 features - F1"] - compare_df["3  
features - F1"]  
display(compare_df.style.format("{:.3f}"))
```

Nhận xét: Chênh lệch F1-score rất nhỏ (~0.004). 6 features có Recall và F1 nhỉnh hơn nhẹ, nhưng nếu ưu tiên giao diện đơn giản thì 3 features cũng là lựa chọn hợp lý.

18. Mô hình cuối cùng

KNN với K tối ưu được chọn làm model cuối cùng. Model được huấn luyện trên toàn bộ dữ liệu và lưu diabetes.sav.

```
import pickle

# Huấn luyện KNN cuối cùng với K tối ưu trên toàn bộ dữ liệu
final_model = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=optimal_k))
])
final_model.fit(X_raw, y)
print(f"Đã huấn luyện model cuối (KNN, K={optimal_k}) trên toàn bộ {len(X_raw)} mẫu")

# Lưu model
filename = "diabetes.sav"
with open(filename, "wb") as model_file:
    pickle.dump(final_model, model_file)
print(f"Đã lưu model: {filename}")
```

-Test dự đoán mẫu và confidence

```
# Test với mẫu dữ liệu
with open("diabetes.sav", "rb") as model_file:
    loaded_model = pickle.load(model_file)

sample = pd.DataFrame([{"Glucose": 65, "BMI": 70, "Age": 50}])
prediction = loaded_model.predict(sample)
probabilities = loaded_model.predict_proba(sample)
confidence = np.max(probabilities[0]) * 100

print(f"Prediction: {prediction[0]} ({'Tiểu đường' if prediction[0] == 1 else 'Không tiểu đường'})")
print(f"Confidence: {confidence:.2f}%")
```

19. Ứng dụng

Model được triển khai thành REST API bằng Flask (REST_API.py). API nhận JSON (Glucose, BMI, Age), trả về prediction (0/1) và confidence (%).

-Triển khai thành REST API

```
import os
import pickle
from collections import defaultdict
from pathlib import Path
from flask import Flask, jsonify, render_template, request
from neo4j import GraphDatabase
import numpy as np
```

```

app = Flask(__name__)

# --- Models ---
filename = "diabetes.sav"
try:
    loaded_model = pickle.load(open(filename, "rb"))
except FileNotFoundError:
    loaded_model = None

# --- Neo4j Config ---
NEO4J_URI = os.getenv('NEO4J_URI', 'bolt://localhost:7687')
NEO4J_USERNAME = os.getenv('NEO4J_USERNAME', 'neo4j')
NEO4J_PASSWORD = os.getenv('NEO4J_PASSWORD')
NEO4J_DATABASE = os.getenv('NEO4J_DATABASE', 'neo4j')

neo4j_driver = (
    GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD))
    if NEO4J_PASSWORD
    else None
)

KNOWLEDGE_GROUPS = {
    'HAS_DIET_ADVICE': 'Chế độ ăn',
    'HAS_LIFESTYLE_ACTION': 'Lối sống và vận động',
    'HAS_DRUG_INFO': 'Thông tin thuốc',
    'HAS_COMPLICATION': 'Biến chứng cần lưu ý',
}

def get_diabetes_knowledge():
    """Read grouped diabetes knowledge and article sources from Neo4j."""
    if neo4j_driver is None:
        return [], 'Neo4j password has not been configured (NEO4J_PASSWORD).'
    query = """
    MATCH (d:Disease {id: 'diabetes'})-
    [relation:HAS_DIET_ADVICE|HAS_LIFESTYLE_ACTION|HAS_DRUG_INFO|HAS_COMPLICATION]-
    >(item)
    OPTIONAL MATCH (item)-[:SOURCED_FROM]->(article:Article)
    RETURN type(relation) AS relation_type,
           coalesce(item.title, item.name) AS title,
           item.content AS content,
           item.duration AS duration,
           item.frequency AS frequency,
           collect(DISTINCT {title: article.title, url: article.url}) AS
sources
           ORDER BY relation_type, title
    """
    try:

```

```

        grouped_items = defaultdict(list)
        with neo4j_driver.session(database=NEO4J_DATABASE) as session:
            for record in session.run(query):
                sources = [s for s in record['sources'] if s.get('title')]
                grouped_items[record['relation_type']].append({
                    'title': record['title'],
                    'content': record['content'],
                    'duration': record['duration'],
                    'frequency': record['frequency'],
                    'sources': sources
                })

        result = []
        for rel_type, label in KNOWLEDGE_GROUPS.items():
            if rel_type in grouped_items:
                result.append({'group_label': label, 'items':
grouped_items[rel_type]})
        return result, None
    except Exception as e:
        return [], str(e)

@app.route("/", methods=["GET"])
def home():
    # If index.html exists, render it for full UI
    if Path("templates/index.html").exists():
        return render_template("index.html")
    return "Diabetes Prediction API is running (Neo4j integrated)"

@app.route("/diabetes/v1/predict", methods=["POST"])
def predict():
    if not loaded_model:
        return jsonify({"error": "Model not found."}), 500

    features = request.json
    try:
        features_list = [
            float(features["Glucose"]),
            float(features["BMI"]),
            float(features["Age"]),
        ]
    except (KeyError, ValueError):
        return jsonify({"error": "Missing or invalid required fields (Glucose,
BMI, Age)"}), 400

    prediction = loaded_model.predict([features_list])
    confidence = loaded_model.predict_proba([features_list])

```

```

has_diabetes = int(prediction[0]) == 1

knowledge, knowledge_error = [], None
if has_diabetes:
    knowledge, knowledge_error = get_diabetes_knowledge()

response = {
    "prediction": int(prediction[0]),
    "confidence": str(round(np.amax(confidence[0]) * 100, 2)),
    "knowledge": knowledge,
    "knowledge_error": knowledge_error
}

return jsonify(response)

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=8080,
        debug=True,
        use_reloader=False,
    )

```

-Gọi API từ notebook

```

import requests

def predict_diabetes(BMI, Age, Glucose):
    url = "http://127.0.0.1:8080/diabetes/v1/predict"
    data = {"BMI": BMI, "Age": Age, "Glucose": Glucose}
    response = requests.post(url, json=data, timeout=10)
    response.raise_for_status()
    return response.json()

```

20. Minh hoạ hệ thống

-Giao diện web desktop

Dự đoán nguy cơ tiểu đường

Nhập các chỉ số lâm sàng cơ bản để hệ thống AI dự đoán nguy cơ và cung cấp lời khuyên y khoa cá nhân hoá.

Mô hình AI dự đoán (Model)

K-Nearest Neighbors

Pregnancies (Số lần mang thai)

3

Glucose (Đường huyết)

150

BMI (Chỉ số khối cơ thể)

32

Age (Tuổi)

25

Skin Thickness (Độ dày da)

24

Insulin

110

Phân tích kết quả

TIN VUI: KHÔNG CÓ NGUY CƠ TIỂU ĐƯỜNG

Model phân tích: **K-Nearest Neighbors**

Độ tin cậy: **66.67%**

-Giao diện web mobile

Dự đoán nguy cơ tiểu đường

Nhập các chỉ số lâm sàng cơ bản để hệ thống AI dự đoán nguy cơ và cung cấp lời khuyên y khoa cá nhân hoá.

Mô hình AI dự đoán (Model)

Logistic Regression



Pregnancies (Số lần mang thai)

3

Glucose (Đường huyết)

150

BMI (Chỉ số khối cơ thể)

32

Age (Tuổi)

25

Skin Thickness (Độ dày da)

24

21. Nhận xét

Bài toán dự đoán nguy cơ tiểu đường được xây dựng từ bộ dữ liệu Pima Indians Diabetes. Các giá trị 0 không hợp lý đã được xem là giá trị thiếu và thay thế.

Dữ liệu có sự mất cân bằng giữa hai nhãn, do đó Accuracy không được sử dụng riêng lẻ. Precision, Recall và F1-score được ưu tiên.

Bốn mô hình đã được huấn luyện và so sánh. KNN dẫn đầu ở cả 4 metric. Thử nghiệm với 6 features cho thấy chênh lệch rất nhỏ so với 3 features.

Hạn chế: bộ dữ liệu nhỏ (768 mẫu), chỉ đại diện cho nhóm phụ nữ Pima Indians, không thể thay thế chẩn đoán y khoa.

22. Kết luận

Notebook đã xây dựng hoàn chỉnh hệ thống dự đoán nguy cơ tiểu đường: thu thập dữ liệu, tiền xử lý, phân tích đặc trưng, chia train/test, baseline, huấn luyện và đánh giá nhiều mô hình.

Bốn mô hình Logistic Regression, KNN, SVM Linear, SVM RBF đã được thử nghiệm. KNN với K tối ưu được chọn làm model cuối cùng và lưu .sav để tích hợp REST API.

Hướng cải tiến: dữ liệu lớn hơn, tối ưu hyperparameter sâu hơn, hiệu chỉnh ngưỡng dự đoán để tăng Recall, triển khai lên cloud.

PHẦN II - HỆ THỐNG DỰ ĐOÁN GIÁ NHÀ

1. System and Problem Definition

1.1. Bối cảnh

Giá nhà phụ thuộc đồng thời vào đặc điểm vật lý, tình trạng pháp lý, nội thất và nhiều yếu tố thị trường. Hệ thống xây dựng dự đoán giá nhà từ thông tin cấu trúc trong bộ dữ liệu Vietnam Housing 2024.

1.2. Phát biểu bài toán

Bài toán hồi quy có giám sát. Đầu vào: Floors, Bedrooms, Bathrooms, Legal status, Furniture state. Đầu ra: Price. Tiêu chí: MAE, RMSE nhỏ; R^2 lớn.

1.3. Mục tiêu và phạm vi

So sánh 4 mô hình hồi quy (Linear Regression, Decision Tree, Random Forest, SVR), chọn model phù hợp và triển khai Flask API. Chỉ hỗ trợ tham khảo, không thay thế định giá chuyên gia.

2. Intelligent System Diagram

2.1. Kiến trúc tổng thể

GIAI ĐOẠN HUẤN LUYỆN: Dataset → EDA → Chọn feature → Chia Train/Test → Pipeline (Numeric: median+StandardScaler, Category: Unknown+OneHot) → 4 models → Đánh giá → Lưu .sav

GIAI ĐOẠN TRIỂN KHAI: Người dùng → Flask API → Load model → Pipeline → Giá dự đoán

GIAI ĐOẠN HUẤN LUYỆN (offline)

DATA/house_prices.csv



Khám phá dữ liệu: kiểu dữ liệu, null, phân phối, tương quan



Chọn biểu diễn: Floors, Bedrooms, Bathrooms,
Legal status, Furniture state



Chia Train / Test (cố định random_state = 42)



Pipeline tiền xử lý

- └ Numeric: median imputation → StandardScaler
- └ Category: "Unknown" imputation → One-Hot Encoding



4 mô hình hồi quy → MAE, RMSE, R^2 → chọn Final Model



models/*.sav

GIAI ĐOẠN TRIỂN KHAI (online)

Người dùng (trang web) → Flask REST API → model .sav / Pipeline



Giá nhà dự đoán (Price)

2.2. Luồng dự đoán

Người dùng nhập thông số nhà vào form web. API nhận dữ liệu, xử lý qua pipeline, đưa vào model để dự đoán giá.

3. Dataset Source

Nội dung	Thông tin
Tên	Vietnam Housing Dataset 2024
Nguồn	Kaggle
Tệp	data/house_price/vietnam_housing_dataset.csv
Quy mô	30,229 dòng, 12 cột
Miền	Bất động sản; hồi quy dự đoán giá nhà

4. Dataset Description

4.1. Đơn vị quan sát và biến mục tiêu

Mỗi hàng là một bất động sản. Biến mục tiêu Price là số liên tục → bài toán hồi quy.

4.2. Các nhóm thuộc tính

Nhóm	Cột	Ý nghĩa
Số	Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms	Kích thước/cấu trúc
Phân loại	House direction, Balcony direction, Legal status, Furniture state	Hướng/pháp lý/nội thất
Vị trí	Address	Địa chỉ
Mục tiêu	Price	Giá nhà

4.3. Chất lượng dữ liệu

Tỷ lệ thiếu cao: Balcony direction ~82.6%, House direction ~70%, Frontage ~38%, Access Road ~44%. Cột số impute bằng median, cột phân loại impute bằng mode/Unknown.

4.4. Cơ sở lý thuyết các mô hình hồi quy

1. Linear Regression — $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$, OLS. Mốc tuyến tính.
2. Decision Tree — chia dữ liệu theo if-else. Phi tuyến, dễ overfit.
3. Random Forest — trung bình nhiều cây, giảm variance.
4. SVR — SVM cho hồi quy, kernel RBF.

4.5. Chỉ số đánh giá hồi quy

MAE = trung bình $|y - \hat{y}|$. RMSE = $\sqrt{\text{MSE}}$. R^2 = tỷ lệ variance giải thích được (0→1).

5. Data Representation

5.1. Kiểm tra dữ liệu

```
# Đọc dữ liệu
df = pd.read_csv('/Users/mac/Docs/Kì 1 năm
4/PTHTTM/data/house_price/vietnam_housing_dataset.csv')
df_raw = df.copy()

print("Shape of dataset:", df.shape)
df.head()
```

Shape of dataset: (30229, 12)

	Address	Area	Frontage	Access Road	House direction	Balcony direction	Floors	Bedrooms	Bathrooms	Legal status	Furniture state	Price
0	Dự án The Empire - Vinhomes Ocean Park 2, Xã L...	84.0	NaN	NaN	NaN	NaN	4.0	NaN	NaN	Have certificate	NaN	8.60
1	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	60.0	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN	7.50
2	Dự án The Crown - Vinhomes Ocean Park 3, Xã Ng...	90.0	6.0	13.0	Đông - Bắc	Đông - Bắc	5.0	NaN	NaN	Sale contract	NaN	8.90
3	Đường Nguyễn Văn Khối, Phường 11, Gò Vấp, Hồ C...	54.0	NaN	3.5	Tây - Nam	Tây - Nam	2.0	2.0	3.0	Have certificate	Full	5.35
4	Đường Quang Trung, Phường 8, Gò Vấp, Hồ Chí Minh	92.0	NaN	NaN	Đông - Nam	Đông - Nam	2.0	4.0	4.0	Have certificate	Full	6.90

```
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 30229 entries, 0 to 30228
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Address               30229 non-null  str
1   Area                  30229 non-null  float64
2   Frontage              18665 non-null  float64
3   Access Road           16932 non-null  float64
4   House direction       8990 non-null   str
5   Balcony direction     5246 non-null   str
6   Floors                26626 non-null  float64
7   Bedrooms              25067 non-null  float64
8   Bathrooms             23155 non-null  float64
9   Legal status          25723 non-null  str
10  Furniture state        16110 non-null  str
11  Price                 30229 non-null  float64
dtypes: float64(7), str(5)
memory usage: 2.8 MB

```

5.2. Kiểm tra chất lượng dữ liệu

```

# Kiểm tra tỷ lệ null
null_pct = (df.isnull().sum() / len(df) * 100).sort_values(ascending=False)
null_report = pd.DataFrame({
    'Số lượng null': df.isnull().sum().sort_values(ascending=False),
    'Tỷ lệ thiếu (%)': null_pct
})
display(null_report[null_report['Số lượng null'] > 0])

```

	Số lượng null	Tỷ lệ thiếu (%)
Balcony direction	24983	82.645804
House direction	21239	70.260346
Furniture state	14119	46.706805
Access Road	13297	43.987562
Frontage	11564	38.254656
Bathrooms	7074	23.401370
Bedrooms	5162	17.076317
Legal status	4506	14.906216
Floors	3603	11.919018

6. Feature and Target Analysis

```
# Thống kê mô tả cho các cột số
num_cols_all = ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms',
                'Bathrooms', 'Price']
for col in num_cols_all:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

display(df[num_cols_all].describe().T)

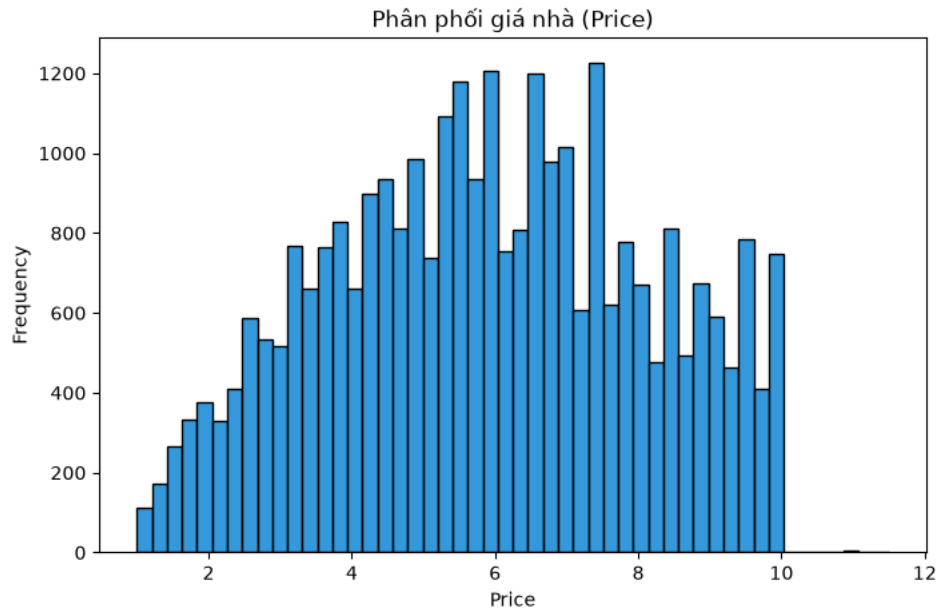
# Thống kê mô tả cho các cột số
num_cols_all = ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms',
                'Bathrooms', 'Price']
for col in num_cols_all:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

display(df[num_cols_all].describe().T)
```

-Biểu đồ phân phối Price

```
# Phân phối biến mục tiêu Price
plt.figure(figsize=(8, 5))
plt.hist(df['Price'].dropna(), bins=50, edgecolor='black', color='#3498db')
plt.title('Phân phối giá nhà (Price)')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```

```
print(f"Price - Mean: {df['Price'].mean():.2f}, Median: {df['Price'].median():.2f}")
print(f"Price - Min: {df['Price'].min():.2f}, Max: {df['Price'].max():.2f}")
```



7. Exploratory Data Analysis

-3 biểu đồ phân phối (Price, Area, Bedrooms)

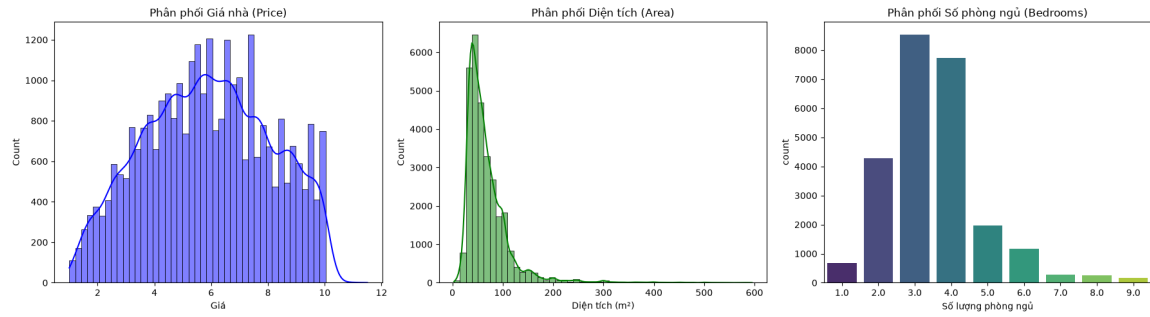
```
# 3 biểu đồ phân phối
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Biểu đồ 1: Phân phối Giá nhà
sns.histplot(df['Price'].dropna(), bins=50, kde=True, ax=axes[0], color='blue')
axes[0].set_title('Phân phối Giá nhà (Price)')
axes[0].set_xlabel('Giá')

# Biểu đồ 2: Phân phối Diện tích
sns.histplot(df['Area'].dropna(), bins=50, kde=True, ax=axes[1], color='green')
axes[1].set_title('Phân phối Diện tích (Area)')
axes[1].set_xlabel('Diện tích (m²)')

# Biểu đồ 3: Phân phối Số phòng ngủ
sns.countplot(x='Bedrooms', data=df.dropna(subset=['Bedrooms']), ax=axes[2],
palette='viridis')
axes[2].set_title('Phân phối Số phòng ngủ (Bedrooms)')
axes[2].set_xlabel('Số lượng phòng ngủ')

plt.tight_layout()
plt.show()
```

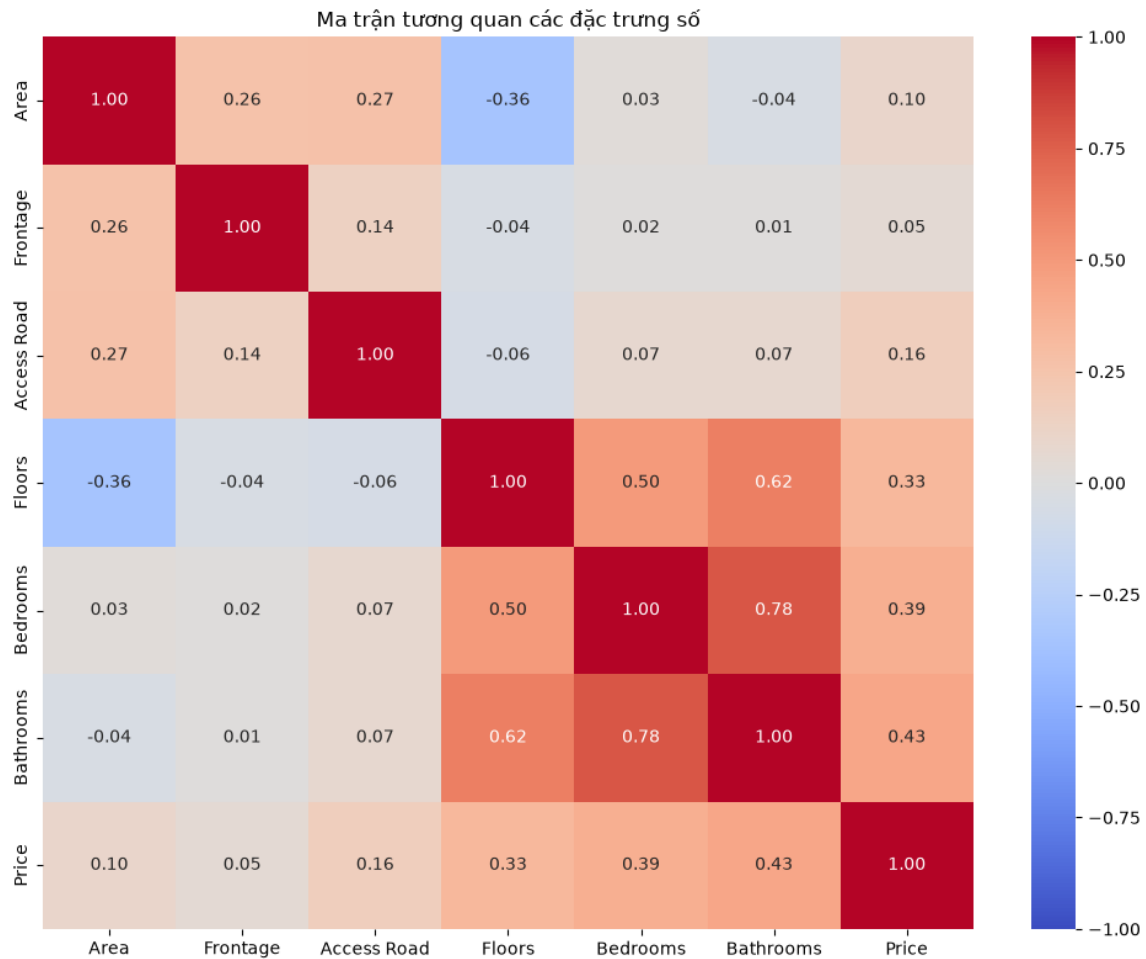


-Correlation Heatmap

```
# Ma trận tương quan cho các cột số
corr_cols = ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms',
             'Bathrooms', 'Price']
corr_matrix = df[corr_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1,
            vmax=1)
plt.title("Ma trận tương quan các đặc trưng số")
plt.tight_layout()
plt.show()

print("\nTương quan với Price:")
print(corr_matrix['Price'].sort_values(ascending=False))
```



```
Tương quan với Price:
Price      1.000000
Bathrooms  0.434101
Bedrooms   0.389432
Floors      0.331639
Access Road 0.164280
Area        0.097593
Frontage    0.047469
Name: Price, dtype: float64
```

=> Bathrooms, Bedrooms và Floors có quan hệ dương rõ nhất với Price.

8. Train/Test Split

Chọn 3 numeric features (Floors, Bedrooms, Bathrooms). Chia 80/20, random_state=42.

Loại outlier (Price > quantile 99%).

```
# Tiền xử lý dữ liệu
df_processed = df.copy()

# Xử lý cột số
num_cols = ['Floors', 'Bedrooms', 'Bathrooms']
for col in num_cols:
    if col in df_processed.columns:
        df_processed[col] = pd.to_numeric(df_processed[col], errors='coerce')
        df_processed[col] =
df_processed[col].fillna(df_processed[col].median())

# Biến mục tiêu
df_processed['Price'] = pd.to_numeric(df_processed['Price'], errors='coerce')
df_processed = df_processed.dropna(subset=['Price'])

# Loại bỏ outlier (quantile 99%)
q = df_processed['Price'].quantile(0.99)
df_processed = df_processed[df_processed['Price'] < q]
print(f"Số mẫu sau khi loại outlier: {len(df_processed)}")

# Chọn feature ban đầu: 3 numeric
NUMERIC_FEATURES = ['Floors', 'Bedrooms', 'Bathrooms']
X = df_processed[NUMERIC_FEATURES].copy()
y = df_processed['Price'].copy()

# Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)
print(f"Train size: {X_train.shape}; Test size: {X_test.shape}")
```

8.1. Xử lý giá trị thiếu sau khi chia

Pipeline: SimpleImputer(median) → StandardScaler. Fit trên train, transform cả train và test.

```
# Pipeline tiền xử lý cho numeric features
from sklearn.pipeline import make_pipeline

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
```

```

])

X_train_processed = numeric_pipeline.fit_transform(X_train)
X_test_processed = numeric_pipeline.transform(X_test)
print("Tiền xử lý hoàn tất.")

```

9. Baseline

DummyRegressor(strategy="mean") luôn dự đoán giá trung bình. $R^2 = 0$.

```

from sklearn.dummy import DummyRegressor

baseline = DummyRegressor(strategy="mean")
baseline.fit(X_train_processed, y_train)
baseline_pred = baseline.predict(X_test_processed)

baseline_mae = mean_absolute_error(y_test, baseline_pred)
baseline_rmse = np.sqrt(mean_squared_error(y_test, baseline_pred))
baseline_r2 = r2_score(y_test, baseline_pred)

print(f"Baseline (Mean) - MAE: {baseline_mae:.3f}, RMSE: {baseline_rmse:.3f},
R²: {baseline_r2:.3f}")

```

10. Model 1 — Linear Regression

```

from sklearn.linear_model import LinearRegression

results = []

lr_model = LinearRegression()
lr_model.fit(X_train_processed, y_train)
lr_train_pred = lr_model.predict(X_train_processed)
lr_test_pred = lr_model.predict(X_test_processed)

# Metrics
lr_metrics = pd.DataFrame({
    "Train": [
        mean_absolute_error(y_train, lr_train_pred),
        np.sqrt(mean_squared_error(y_train, lr_train_pred)),
        r2_score(y_train, lr_train_pred)
    ],
    "Test": [
        mean_absolute_error(y_test, lr_test_pred),
        np.sqrt(mean_squared_error(y_test, lr_test_pred)),

```

```

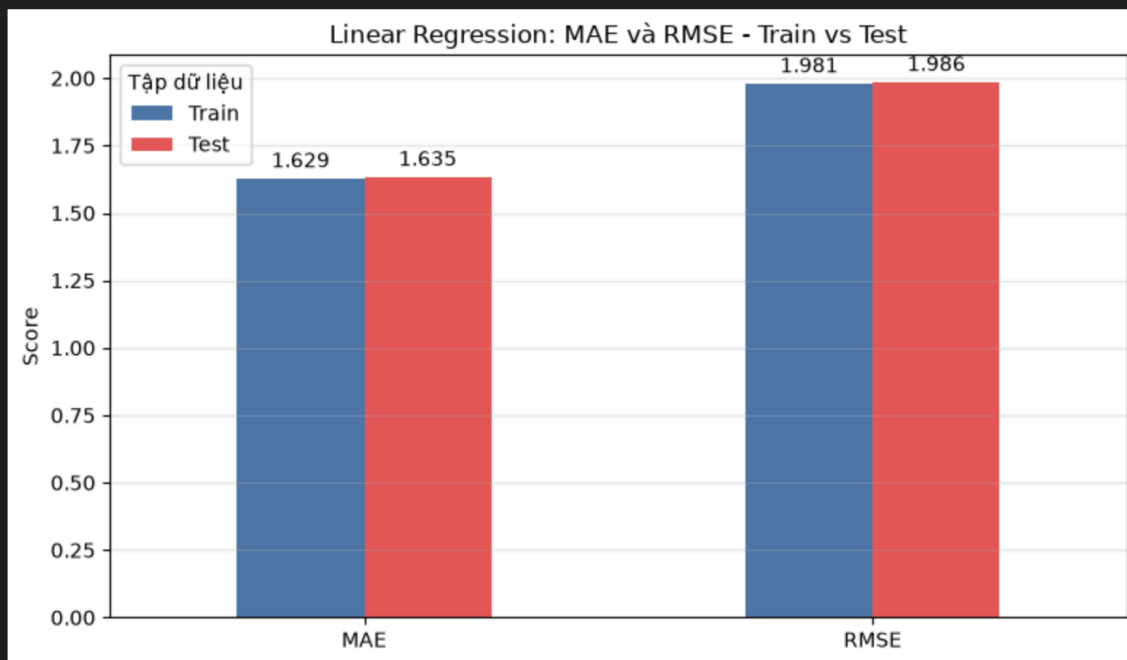
        r2_score(y_test, lr_test_pred)
    ]
}, index=["MAE", "RMSE", "R²"])
display(lr_metrics)

results.append({
    'Model': 'Linear Regression',
    'MAE': mean_absolute_error(y_test, lr_test_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, lr_test_pred)),
    'R²': r2_score(y_test, lr_test_pred)
})

# Biểu đồ Train vs Test
ax = lr_metrics.loc[["MAE", "RMSE"]].plot(kind="bar", figsize=(9, 5),
color=["#4C78A8", "#E45756"])
plt.title("Linear Regression: MAE và RMSE - Train vs Test")
plt.ylabel("Score"); plt.xticks(rotation=0)
plt.legend(title="Tập dữ liệu"); plt.grid(axis="y", alpha=0.3)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

```

	Train	Test
MAE	1.628758	1.634773
RMSE	1.980886	1.986469
R²	0.162212	0.159731



11. Model 2 — Decision Tree Regressor

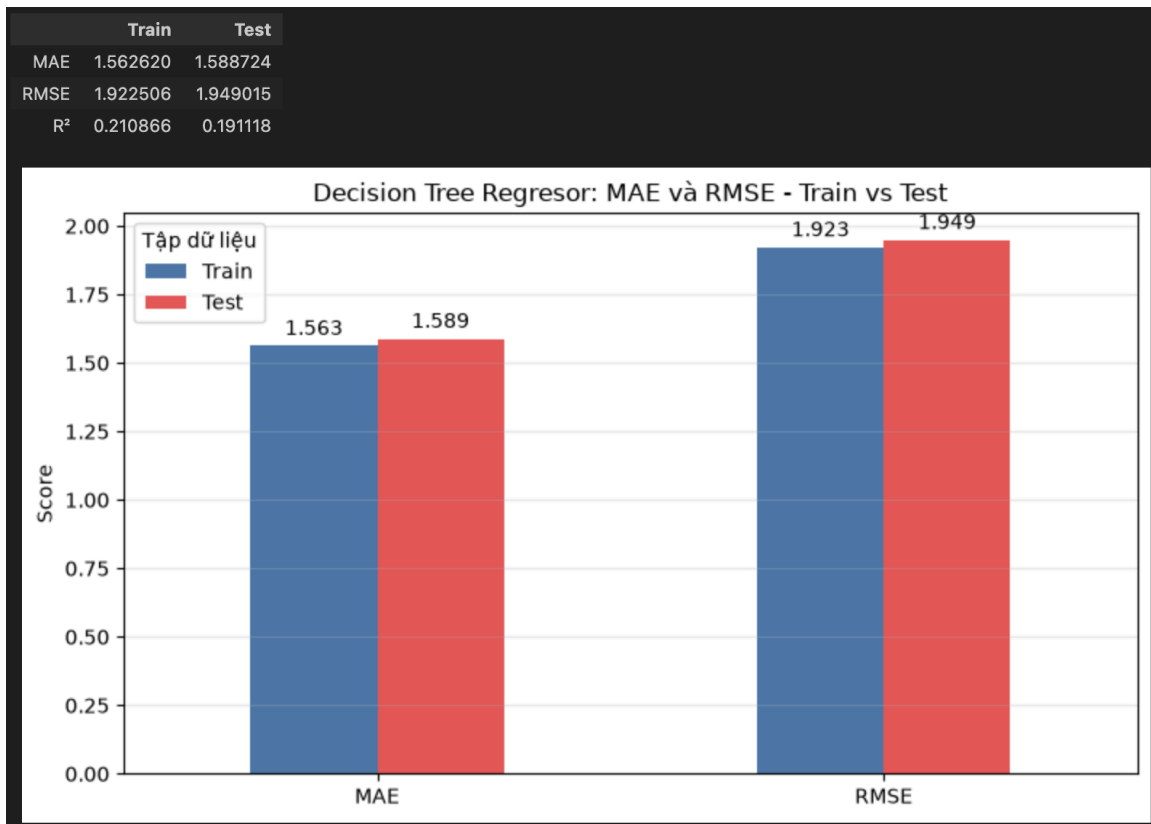
```
from sklearn.tree import DecisionTreeRegressor

dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train_processed, y_train)
dt_train_pred = dt_model.predict(X_train_processed)
dt_test_pred = dt_model.predict(X_test_processed)

dt_metrics = pd.DataFrame({
    "Train": [
        mean_absolute_error(y_train, dt_train_pred),
        np.sqrt(mean_squared_error(y_train, dt_train_pred)),
        r2_score(y_train, dt_train_pred)
    ],
    "Test": [
        mean_absolute_error(y_test, dt_test_pred),
        np.sqrt(mean_squared_error(y_test, dt_test_pred)),
        r2_score(y_test, dt_test_pred)
    ]
}, index=["MAE", "RMSE", "R²"])
display(dt_metrics)

results.append({
    'Model': 'Decision Tree',
    'MAE': mean_absolute_error(y_test, dt_test_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, dt_test_pred)),
    'R²': r2_score(y_test, dt_test_pred)
})

# Biểu đồ Train vs Test
ax = dt_metrics.loc[["MAE", "RMSE"]].plot(kind="bar", figsize=(9, 5),
color=["#4C78A8", "#E45756"])
plt.title("Decision Tree Regresor: MAE và RMSE – Train vs Test")
plt.ylabel("Score"); plt.xticks(rotation=0)
plt.legend(title="Tập dữ liệu"); plt.grid(axis="y", alpha=0.3)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()
```



12. Model 3 — Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_processed, y_train)
rf_train_pred = rf_model.predict(X_train_processed)
rf_test_pred = rf_model.predict(X_test_processed)

rf_metrics = pd.DataFrame({
    "Train": [
        mean_absolute_error(y_train, rf_train_pred),
        np.sqrt(mean_squared_error(y_train, rf_train_pred)),
        r2_score(y_train, rf_train_pred)
    ],
    "Test": [
        mean_absolute_error(y_test, rf_test_pred),
        np.sqrt(mean_squared_error(y_test, rf_test_pred)),
        r2_score(y_test, rf_test_pred)
    ]
}, index=["MAE", "RMSE", "R2"])
display(rf_metrics)
```

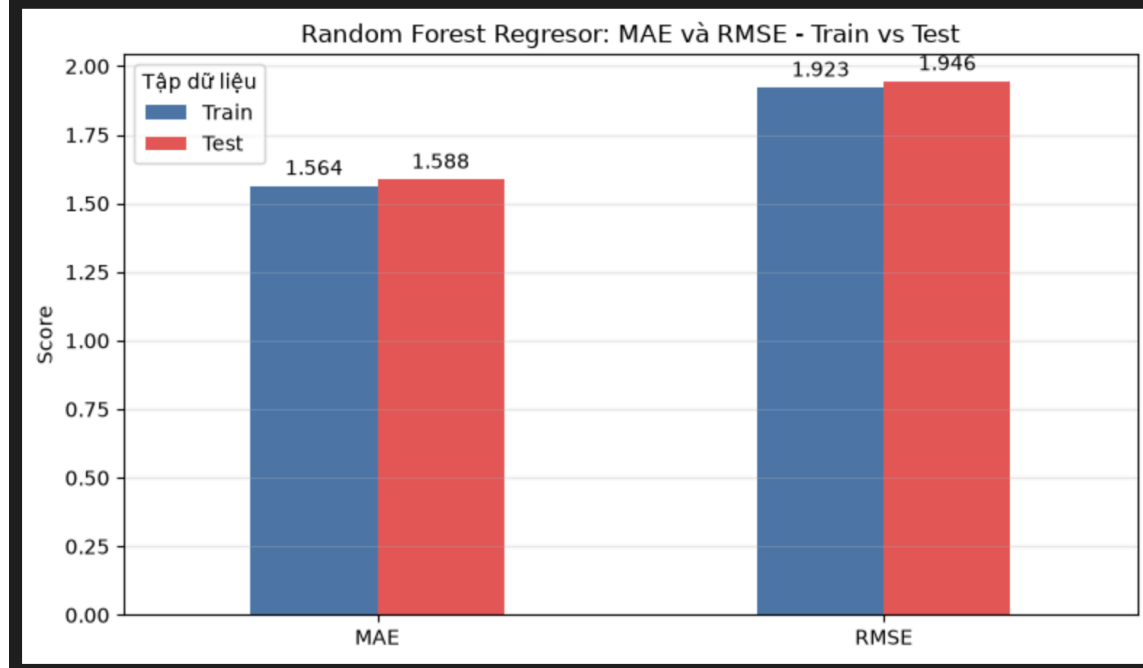
```

results.append({
    'Model': 'Random Forest',
    'MAE': mean_absolute_error(y_test, rf_test_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, rf_test_pred)),
    'R²': r2_score(y_test, rf_test_pred)
})

# Biểu đồ Train vs Test
ax = rf_metrics.loc[["MAE", "RMSE"]].plot(kind="bar", figsize=(9, 5),
color=["#4C78A8", "#E45756"])
plt.title("Random Forest Regresor: MAE và RMSE - Train vs Test")
plt.ylabel("Score"); plt.xticks(rotation=0)
plt.legend(title="Tập dữ liệu"); plt.grid(axis="y", alpha=0.3)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

```

	Train	Test
MAE	1.564244	1.587918
RMSE	1.922995	1.945525
R²	0.210465	0.194012



13. Model 4 — Support Vector Regression (SVR)

```

from sklearn.svm import SVR

svr_model = SVR(kernel='rbf')
svr_model.fit(X_train_processed, y_train)

```

```

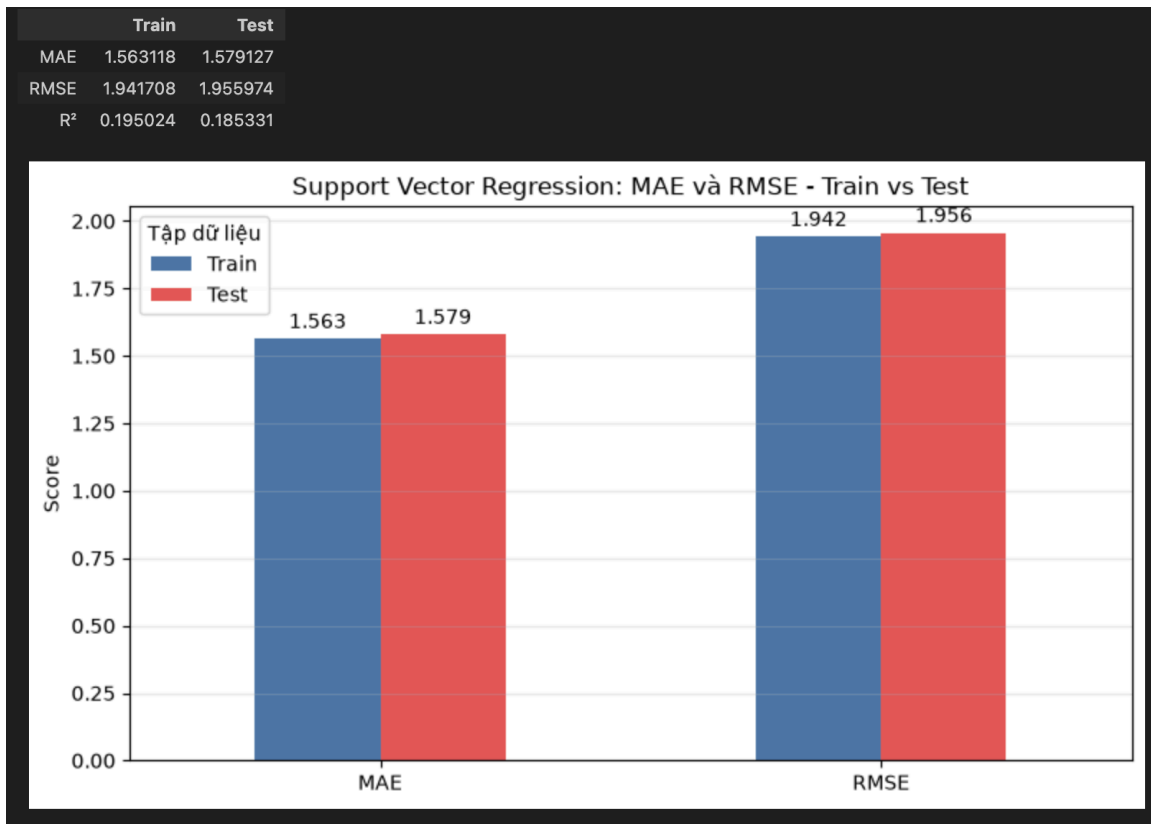
svr_train_pred = svr_model.predict(X_train_processed)
svr_test_pred = svr_model.predict(X_test_processed)

svr_metrics = pd.DataFrame({
    "Train": [
        mean_absolute_error(y_train, svr_train_pred),
        np.sqrt(mean_squared_error(y_train, svr_train_pred)),
        r2_score(y_train, svr_train_pred)
    ],
    "Test": [
        mean_absolute_error(y_test, svr_test_pred),
        np.sqrt(mean_squared_error(y_test, svr_test_pred)),
        r2_score(y_test, svr_test_pred)
    ]
}, index=["MAE", "RMSE", "R2"])
display(svr_metrics)

results.append({
    'Model': 'SVR',
    'MAE': mean_absolute_error(y_test, svr_test_pred),
    'RMSE': np.sqrt(mean_squared_error(y_test, svr_test_pred)),
    'R2': r2_score(y_test, svr_test_pred)
})

# Biểu đồ Train vs Test
ax = svr_metrics.loc[["MAE", "RMSE"]].plot(kind="bar", figsize=(9, 5),
color=["#4C78A8", "#E45756"])
plt.title("Support Vector Regression: MAE và RMSE – Train vs Test")
plt.ylabel("Score"); plt.xticks(rotation=0)
plt.legend(title="Tập dữ liệu"); plt.grid(axis="y", alpha=0.3)
for container in ax.containers:
    ax.bar_label(container, fmt="%.3f", padding=3)
plt.show()

```



14. Evaluation

```
# Bảng so sánh
results_df = pd.DataFrame(results).set_index('Model')
results_df = results_df.sort_values("R2", ascending=False)
display(results_df.style.format("{:.3f}"))
```

	MAE	RMSE	R ²
Model			
Random Forest	1.588	1.946	0.194
Decision Tree	1.589	1.949	0.191
SVR	1.579	1.956	0.185
Linear Regression	1.635	1.986	0.160

15. Experiment 1: Model Comparison

```
# Biểu đồ so sánh MAE, RMSE
```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

results_df["MAE"].plot(kind="bar", ax=axes[0], color="#4C78A8")
axes[0].set_title("MAE theo Model"); axes[0].set_ylabel("MAE")
axes[0].tick_params(axis='x', rotation=0)

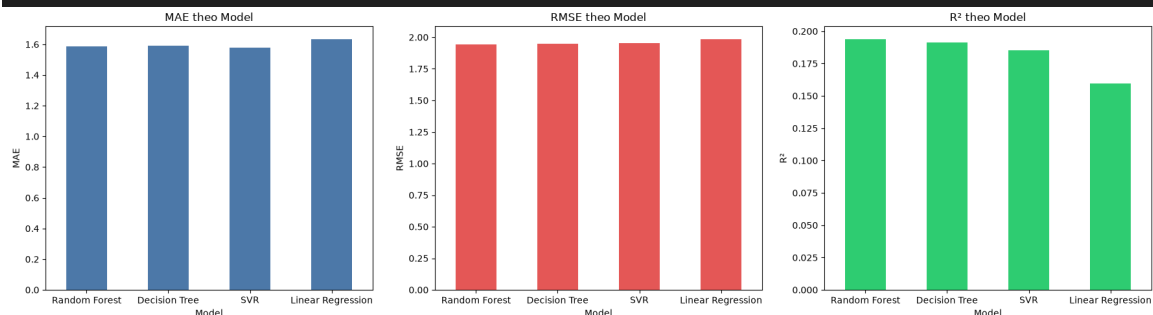
results_df["RMSE"].plot(kind="bar", ax=axes[1], color="#E45756")
axes[1].set_title("RMSE theo Model"); axes[1].set_ylabel("RMSE")
axes[1].tick_params(axis='x', rotation=0)

results_df["R²"].plot(kind="bar", ax=axes[2], color="#2ecc71")
axes[2].set_title("R² theo Model"); axes[2].set_ylabel("R²")
axes[2].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

# Xác định model tốt nhất
for metric in ["MAE", "RMSE", "R²"]:
    if metric == "R²":
        best = results_df[metric].idxmax()
    else:
        best = results_df[metric].idxmin()
    print(f"{metric} tốt nhất: {best} ({results_df.loc[best, metric]:.3f})")

```



=>MAE tốt nhất: SVR (1.579)

RMSE tốt nhất: Random Forest (1.946)

R² tốt nhất: Random Forest (0.194)

16. Experiment 2: Hyperparameter Investigation

-Khảo sát max_depth cho Decision Tree

```

# Khảo sát max_depth cho Decision Tree
from sklearn.model_selection import cross_val_score

depths = list(range(1, 21))
cv_scores_dt = []

```

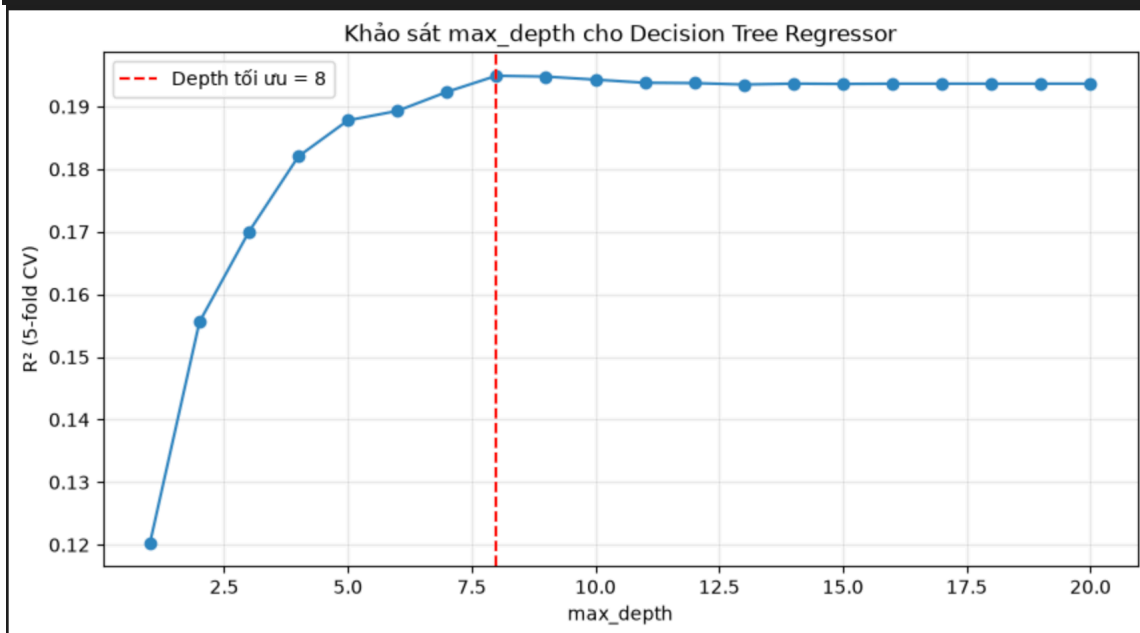
```

for d in depths:
    dt = DecisionTreeRegressor(max_depth=d, random_state=42)
    score = cross_val_score(dt, X_train_processed, y_train, cv=5,
scoring="r2").mean()
    cv_scores_dt.append(score)

optimal_depth = depths[cv_scores_dt.index(max(cv_scores_dt))]
print(f"Max_depth tối ưu cho Decision Tree: {optimal_depth}")
print(f"R2 CV tốt nhất: {max(cv_scores_dt):.4f}")

plt.figure(figsize=(10, 5))
plt.plot(depths, cv_scores_dt, marker="o", color="#2E86C1")
plt.axvline(optimal_depth, color="red", linestyle="--", label=f"Depth tối ưu = {optimal_depth}")
plt.title("Khảo sát max_depth cho Decision Tree Regressor")
plt.xlabel("max_depth"); plt.ylabel("R2 (5-fold CV)")
plt.grid(alpha=0.3); plt.legend()
plt.show()

```



=>Max_depth tối ưu cho Decision Tree: 8

R² CV tốt nhất: 0.1949

-Khảo sát n_estimators cho Random Forest

```

# Khảo sát n_estimators cho Random Forest
n_trees = [10, 25, 50, 100, 150, 200]
cv_scores_rf = []

for n in n_trees:
    rf = RandomForestRegressor(n_estimators=n, random_state=42)
    score = cross_val_score(rf, X_train_processed, y_train, cv=5,
scoring="r2").mean()

```

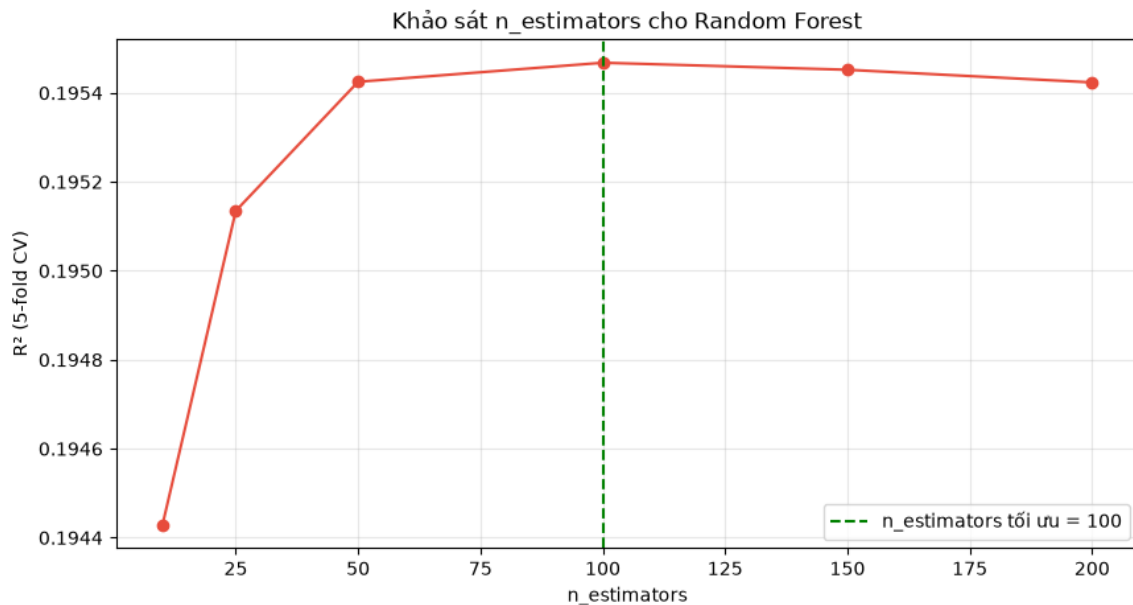
```

cv_scores_rf.append(score)

optimal_n = n_trees[cv_scores_rf.index(max(cv_scores_rf))]
print(f"n_estimators tối ưu cho Random Forest: {optimal_n}")
print(f"R2 CV tốt nhất: {max(cv_scores_rf):.4f}")

plt.figure(figsize=(10, 5))
plt.plot(n_trees, cv_scores_rf, marker="o", color="#e74c3c")
plt.axvline(optimal_n, color="green", linestyle="--", label=f"n_estimators tối ưu = {optimal_n}")
plt.title("Khảo sát n_estimators cho Random Forest")
plt.xlabel("n_estimators"); plt.ylabel("R2 (5-fold CV)")
plt.grid(alpha=0.3); plt.legend()
plt.show()

```



⇒ n_estimators tối ưu cho Random Forest: 100
R² CV tốt nhất: 0.1955

17. Experiment 3: Representation / Feature Investigation

Bổ sung Legal status và Furniture state (3 numeric + 2 category → 5 features).

```

# Bổ sung Legal status và Furniture state
cat_cols = ['Legal status', 'Furniture state']
FEATURE_COLS_5 = ['Floors', 'Bedrooms', 'Bathrooms', 'Legal status', 'Furniture state']

df_exp = df_processed.copy()
for col in cat_cols:
    if col in df_exp.columns:

```

```

        df_exp[col] = df_exp[col].fillna("Unknown")

X_5 = df_exp[FEATURE_COLS_5].copy()
y_5 = df_exp['Price'].copy()

X_train_5, X_test_5, y_train_5, y_test_5 = train_test_split(
    X_5, y_5, test_size=0.20, random_state=42
)

# Pipeline cho mixed features (numeric + category)
numeric_features_5 = ['Floors', 'Bedrooms', 'Bathrooms']
category_features_5 = ['Legal status', 'Furniture state']

preprocessor_5 = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_features_5),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ]), category_features_5)
])

X_train_5_proc = preprocessor_5.fit_transform(X_train_5)
X_test_5_proc = preprocessor_5.transform(X_test_5)
print(f"Shape sau xử lý: Train {X_train_5_proc.shape}, Test {X_test_5_proc.shape}")

```

-Huấn luyện 4 models trên 5 features

```

# Huấn luyện 4 models trên 5 features
results_5 = []

models_5 = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'SVR': SVR(kernel='rbf')
}

for name, model in models_5.items():
    model.fit(X_train_5_proc, y_train_5)
    pred = model.predict(X_test_5_proc)
    results_5.append({
        'Model': name,

```

```

    'MAE': mean_absolute_error(y_test_5, pred),
    'RMSE': np.sqrt(mean_squared_error(y_test_5, pred)),
    'R²': r2_score(y_test_5, pred)
})

results_5_df = pd.DataFrame(results_5).set_index('Model')
results_5_df = results_5_df.sort_values("R²", ascending=False)
display(results_5_df.style.format("{:.3f}"))

```

	MAE	RMSE	R²
Model			
Random Forest	1.586	1.952	0.189
SVR	1.570	1.952	0.189
Decision Tree	1.593	1.964	0.179
Linear Regression	1.625	1.976	0.168

-So sánh 3 features và 5 features

```

# So sánh 3 features vs 5 features
compare_df = pd.DataFrame({
    "3 features R²": results_df["R²"],
    "5 features R²": results_5_df["R²"],
})
compare_df["Chênh lệch"] = compare_df["5 features R²"] - compare_df["3 features R²"]
display(compare_df.style.format("{:.3f}"))

print("\nNhận xét: Bổ sung Legal status và Furniture state cải thiện nhẹ ở tất cả 4 model.")

```

	3 features R²	5 features R²	Chênh lệch
Model			
Decision Tree	0.191	0.179	-0.013
Linear Regression	0.160	0.168	0.008
Random Forest	0.194	0.189	-0.005
SVR	0.185	0.189	0.003

=>Bổ sung Legal status và Furniture state cải thiện nhẹ nhưng nhất quán ở tất cả 4 model.

18. Final Model

Decision Tree Regressor có kết quả tốt nhất nhẹ và được chọn làm model cuối. Cả 4 model được lưu .sav.

```
import pickle

# Chọn model cuối cùng: Decision Tree với 5 features (có R² tốt nhất nhẹ)
final_models = {}
FEATURE_COLUMNS_FINAL = ['Floors', 'Bedrooms', 'Bathrooms', 'Legal status',
                          'Furniture state']

# Huấn luyện trên toàn bộ dữ liệu
X_all = df_exp[FEATURE_COLUMNS_FINAL].copy()
y_all = df_exp['Price'].copy()

preprocessor_final = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), ['Floors', 'Bedrooms', 'Bathrooms']),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ]), ['Legal status', 'Furniture state'])
])

X_all_proc = preprocessor_final.fit_transform(X_all)

# Lưu tất cả 4 models
for name, ModelClass in [
    ('linear_regression', LinearRegression()),
    ('decision_tree', DecisionTreeRegressor(random_state=42)),
    ('random_forest', RandomForestRegressor(n_estimators=100,
random_state=42)),
    ('svr', SVR(kernel='rbf'))
]:
    ModelClass.fit(X_all_proc, y_all)
    final_models[name] = ModelClass

    fname = f"models/{name}.sav"
    import os
    os.makedirs("models", exist_ok=True)
    with open(fname, "wb") as f:
        pickle.dump(ModelClass, f)
    print(f"Đã lưu: {fname}")

# Lưu preprocessor
```

```
with open("models/preprocessor.sav", "wb") as f:
    pickle.dump(preprocessor_final, f)
print("Đã lưu preprocessor.")
```

19. Application

Flask web app (flask_app.py + templates/index.html) cho phép nhập thông tin nhà và nhận giá dự đoán.

```
from flask import Flask, request, render_template, jsonify
import tensorflow as tf
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
import os

app = Flask(__name__)

# 1. Load data and fit Scaler/Encoders to match the training env
print("Loading data and fitting scalers...")
df = pd.read_csv('../data/house_price/vietnam_housing_dataset.csv')
df_processed = df.copy()

num_cols = ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms',
            'Bathrooms']
for col in num_cols:
    if col in df_processed.columns:
        df_processed[col] = pd.to_numeric(df_processed[col], errors='coerce')
        df_processed[col] =
df_processed[col].fillna(df_processed[col].median())

cat_cols = ['House direction', 'Balcony direction', 'Legal status', 'Furniture
state', 'Address']
for col in cat_cols:
    if col in df_processed.columns:
        df_processed[col] =
df_processed[col].fillna(df_processed[col].mode()[0])

le_dict = {}
for col in cat_cols:
    if col in df_processed.columns:
        le = LabelEncoder()
        df_processed[col] = le.fit_transform(df_processed[col].astype(str))
        le_dict[col] = le
```

```

df_processed = df_processed.dropna(subset=['Price'])
df_processed['Price'] = pd.to_numeric(df_processed['Price'], errors='coerce')
df_processed = df_processed.dropna(subset=['Price'])
df_processed = df_processed.fillna(0)
q = df_processed['Price'].quantile(0.99)
df_processed = df_processed[df_processed['Price'] < q]

X = df_processed.drop('Price', axis=1)
scaler = StandardScaler()
scaler.fit(X)
ordered_cols = X.columns
print("Data scaling ready.")

# 2. Load TFLite Model
interpreter = tf.lite.Interpreter(model_path="mobile_house_price_model.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json
    try:
        area = float(data.get('Area', 50))
        frontage = float(data.get('Frontage', 5))
        access = float(data.get('Access Road', 5))
        floors = float(data.get('Floors', 2))
        bed = float(data.get('Bedrooms', 2))
        bath = float(data.get('Bathrooms', 2))

        def encode_cat(col, val):
            le = le_dict.get(col)
            if le is not None:
                if val in le.classes_:
                    return float(le.transform([val])[0])
                else:
                    return 0.0 # fallback
            return 0.0

        h_dir = encode_cat('House direction', data.get('House direction', ''))
        b_dir = encode_cat('Balcony direction', data.get('Balcony direction', ''))

        leg = encode_cat('Legal status', data.get('Legal status', ''))
        fur = encode_cat('Furniture state', data.get('Furniture state', ''))

```

```

        addr = encode_cat('Address', data.get('Address', ''))

        val_map = {
            'Area': area, 'Frontage': frontage, 'Access Road': access,
            'Floors': floors,
            'Bedrooms': bed, 'Bathrooms': bath, 'House direction': h_dir,
            'Balcony direction': b_dir, 'Legal status': leg, 'Furniture state':
fur,
            'Address': addr
        }

        feature_list = [val_map[c] for c in ordered_cols]
        feature_arr = np.array([feature_list], dtype=np.float32)

        # Scale
        scaled_features = scaler.transform(feature_arr).astype(np.float32)

        # Predict
        interpreter.set_tensor(input_details[0]['index'], scaled_features)
        interpreter.invoke()
        output_data = interpreter.get_tensor(output_details[0]['index'])

        predicted_price = float(output_data[0][0])
        return jsonify({'success': True, 'price': round(predicted_price, 2)})
    except Exception as e:
        return jsonify({'success': False, 'error': str(e)})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=False)

```

20. System Demonstration

-Giao diện web desktop

AI Định Giá Bất Động Sản

Nhập thông tin mô tả chi tiết của căn nhà để hệ thống dự báo mức giá phù hợp trên thị trường.

Mô hình AI dự đoán (Model)

Decision Tree Regressor

Số tầng (Floors)

3

Số phòng ngủ (Bedrooms)

3

Số phòng vệ sinh (Bathrooms)

2

Tình trạng pháp lý (Legal status)

Hợp đồng mua bán

Nội thất (Furniture state)

Cơ bản (Basic)

Định giá ngay

Dự báo: 5,6 Tỷ VNĐ

Model áp dụng: **Decision Tree Regressor**

Mức giá trên chỉ mang tính chất dự báo dựa trên dữ liệu lịch sử và không đại diện cho thẩm định giá chính thức.

-Giao diện web mobile

AI Định Giá Bất Động Sản

Nhập thông tin mô tả chi tiết của căn nhà để hệ thống dự báo mức giá phù hợp trên thị trường.

Mô hình AI dự đoán (Model)

Support Vector Regression (SVR) ✓

Số tầng (Floors)

3

Số phòng ngủ (Bedrooms)

3

Số phòng vệ sinh (Bathrooms)

2

Tình trạng pháp lý (Legal status)

Hợp đồng mua bán ✓

Nội thất (Furniture state)

Cơ bản (Basic) ✓

Định giá ngay

21. Reflection

Ma trận tương quan cho thấy Bathrooms, Bedrooms và Floors có quan hệ dương rõ nhất với Price. Experiment 3 cho thấy bổ sung Legal status và Furniture state cải thiện nhẹ nhưng nhất quán.

MAE, RMSE và R^2 cần được xem cùng nhau. Giá nhà thực tế còn phụ thuộc vị trí chi tiết, diện tích, thời điểm giao dịch. Các yếu tố này là hạn chế lớn, giải thích vì sao R^2 vẫn còn thấp.

22. Conclusion

Notebook đã thực hiện quy trình dự đoán giá nhà: mô tả dữ liệu, EDA, chọn feature, train/test, baseline, 4 model hồi quy, đánh giá, so sánh model, khảo sát hyperparameter và feature.

Bộ 5 feature (Floors, Bedrooms, Bathrooms, Legal status, Furniture state) được chọn cho model cuối. Cả 4 model được lưu .sav để tích hợp API.

Hướng cải tiến: bổ sung Area, Address/vị trí, mặt tiền bằng feature engineering phù hợp.